

Circulant matrices: Toeplitz matrices where each column is a circular shift of its preceding column:

$$C = \begin{bmatrix} c_0 & c_{n-1} & \cdots & c_2 & c_1 \\ c_1 & c_0 & c_{n-1} & & c_2 \\ c_2 & c_1 & c_0 & & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots \\ c_{n-2} & & & \ddots & \ddots & c_{n-1} \\ c_{n-1} & c_{n-2} & \cdots & c_2 & c_1 & c_0 \end{bmatrix}.$$

n -Cycle matrix: The $n \times n$ matrix with ones along the subdiagonal and in the 1, n -entry, and zeros elsewhere, i.e.,

$$C_n = \begin{bmatrix} 0 & 0 & \cdots & 0 & 1 \\ 1 & 0 & 0 & & 0 \\ \vdots & 1 & 0 & \ddots & \vdots \\ 0 & & \ddots & \ddots & 0 \\ 0 & 0 & \cdots & 1 & 0 \end{bmatrix}.$$

Hankel matrices: Matrices with constant elements along their antidiagonals, i.e.,

$$H = \begin{bmatrix} h_0 & h_1 & \cdots & h_{n-2} & h_{n-1} \\ h_1 & h_2 & \ddots & \ddots & h_n \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ h_{n-2} & \ddots & \ddots & \ddots & h_{2n-2} \\ h_{n-1} & h_n & \cdots & h_{2n-2} & h_{2n-1} \end{bmatrix}.$$

Anti-identity matrix: The matrix with ones along the antidiagonal and zeros elsewhere, i.e.,

$$P_n = \begin{bmatrix} 0 & 0 & \cdots & 0 & 1 \\ 0 & 0 & \ddots & 1 & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 1 & \ddots & \ddots & 0 \\ 1 & 0 & \cdots & 0 & 0 \end{bmatrix}.$$

Cauchy matrices: Given vectors $\mathbf{x} = [x_1, \dots, x_n]^T$ and $\mathbf{y} = [y_1, \dots, y_n]^T$, the Cauchy matrix $C(\mathbf{x}, \mathbf{y})$ has i, j -entry equal to $\frac{1}{x_i + y_j}$.

Vandermonde matrices: A matrix having each row equal to successive powers of a number, i.e.,

$$V = \begin{bmatrix} 1 & v_1^1 & \cdots & v_1^{n-2} & v_1^{n-1} \\ 1 & v_2^1 & \cdots & v_2^{n-2} & v_2^{n-1} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & v_n^1 & \cdots & v_n^{n-2} & v_n^{n-1} \end{bmatrix}.$$

Block matrices: An $m \times m$ block matrix with $n \times n$ blocks is a matrix of the form

$$\begin{bmatrix} A^{(1,1)} & A^{(1,2)} & \cdots & A^{(1,m)} \\ A^{(2,1)} & A^{(2,2)} & \cdots & A^{(2,m)} \\ \vdots & \vdots & \ddots & \vdots \\ A^{(m-1,1)} & A^{(m-1,2)} & \cdots & A^{(m-1,m)} \\ A^{(m,1)} & A^{(m,2)} & \cdots & A^{(m,m)} \end{bmatrix},$$

where each block $A^{(i,j)}$ is an $n \times n$ matrix.

Toeplitz-block matrices: $mn \times mn$ block matrices where each block $\{A^{(i,j)}\}_{i,j=1}^m$ is an $n \times n$ Toeplitz matrix.

Block-Toeplitz matrices: $mn \times mn$ block matrices of the form

$$T = \begin{bmatrix} A^{(0)} & A^{(-1)} & \cdots & A^{(1-m)} \\ A^{(1)} & A^{(0)} & \cdots & A^{(2-m)} \\ \vdots & \vdots & \ddots & \vdots \\ A^{(m-1)} & A^{(m-2)} & \cdots & A^{(0)} \end{bmatrix},$$

where $\{A^{(i)}\}_{i=1-m}^{m-1}$ are arbitrary $n \times n$ matrices.

Block-Toeplitz–Toeplitz-block (BTTB) matrices: The blocks $A^{(i)}$ are themselves Toeplitz matrices.

Block matrices for other structured matrices such as the block-circulant matrices or the circulant-block matrices can be defined similarly.

Facts:

1. The transpose of a Toeplitz matrix is a Toeplitz matrix.
2. Any linear combination of Toeplitz matrices is a Toeplitz matrix.
3. The lower shift matrices Z_n are Toeplitz matrices. The $n \times n$ Toeplitz matrix $T = [t_{ij}]$ with $t_{ij} = t_{i-j}$ satisfies $T = t_0 I_n + \sum_{k=1}^{n-1} t_k Z_n^k + \sum_{k=1}^{n-1} t_{-k} (Z_n^T)^k$.
4. Every circulant matrix is a Toeplitz matrix, but not conversely.
5. The transpose of a circulant matrix is a circulant matrix.
6. Any linear combination of circulant matrices is a circulant matrix.
7. The n -cycle matrix C_n is a circulant matrix. The $n \times n$ circulant matrix $C = [c_{ij}]$ with $c_{i1} = c_{i-1}$ satisfies $C = \sum_{k=0}^{n-1} c_k C_n^k$.
8. An important property of circulant matrices is that they can be diagonalized by discrete Fourier transform matrices (see Section 47.3 and Chapter 58.3). Thus circulant matrices are normal.
9. A Hankel matrix is symmetric.
10. Any linear combination of Hankel matrices is a Hankel matrix.
11. Multiplication of a Toeplitz matrix and the anti-identity matrix P_n is a Hankel matrix, and multiplication of a Hankel matrix and P_n is a Toeplitz matrix. For H as in the definition,

$$P_n H = \begin{bmatrix} h_{n-1} & h_n & \cdots & h_{2n-2} & h_{2n-1} \\ h_{n-2} & h_{n-1} & h_n & & h_{2n-2} \\ \vdots & h_{n-2} & h_{n-1} & \ddots & \vdots \\ h_1 & & \ddots & \ddots & h_n \\ h_0 & h_1 & \cdots & h_{n-2} & h_{n-1} \end{bmatrix},$$

$$HP_n = \begin{bmatrix} h_{n-1} & h_{n-2} & \cdots & h_1 & h_0 \\ h_n & h_{n-1} & h_{n-2} & & h_1 \\ \vdots & h_n & h_{n-1} & \ddots & \vdots \\ h_{2n-2} & & \ddots & \ddots & h_{-1} \\ h_{2n-1} & h_{2n-2} & \cdots & h_n & h_{n-1} \end{bmatrix}.$$

12. The Kronecker product $A \otimes T$ of any matrix A and a Toeplitz matrix T is a Toeplitz-block matrix and Kronecker product $T \otimes A$ is a block-Toeplitz matrix.
13. Most of the applications, such as partial differential equations and image processing, are concerned [Jin02], [Ng04] with two-dimensional problems where the matrices will have block structures.

Examples:

1. $\begin{bmatrix} 1 & 2 & 1 & 1 \\ 3 & 1 & 1 & 1 \\ -5 & 2 & 1 & 2 \\ 0 & -5 & 3 & 1 \end{bmatrix}$ is a BTTB matrix with $A^{(0)} = \begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}$, $A^{(-1)} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$, $A^{(1)} = \begin{bmatrix} -5 & 2 \\ 0 & 5 \end{bmatrix}$.
2. For $\mathbf{x} = \begin{bmatrix} 1 \\ 1 \\ 5 \end{bmatrix}$ and $\mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$, $C(\mathbf{x}, \mathbf{y}) = \begin{bmatrix} \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{6} & \frac{1}{7} & \frac{1}{8} \end{bmatrix}$.

48.2 Direct Toeplitz Solvers

Most of the early work on Toeplitz solvers was focused on direct methods. These systems arise in a variety of applications in mathematics and engineering. In fact, Toeplitz structure was one of the first structures analyzed in signal processing.

Definitions:

Toeplitz systems: A system of linear equations with a Toeplitz coefficient matrix.

Facts:

1. Given an $n \times n$ Toeplitz system $Tx = b$, a straightforward application of the Gaussian elimination method will result in an algorithm of $O(n^3)$ complexity.
2. However, since the matrix is determined by only $(2n - 1)$ entries rather than n^2 entries, it is to be expected that a solution can be obtained in less than $O(n^3)$ operations. There are a number of fast Toeplitz solvers that can reduce the complexity to $O(n^2)$ operations. The original references for these algorithms are Schur [Sch17], Levinson [Lev46], Durbin [Dur60], and Trench [Tre64].
3. In the 1980s, superfast algorithms of complexity $(n \log^2 n)$ operations for Toeplitz systems were proposed by a different group of researchers: Bitmead and Anderson [BA80], Brent et al. [BGY80], Morf [Mor80], de Hoog [Hoo87], and Ammar and Gragg [AG88]. The key to these direct methods is to solve the system recursively. In this section, we will give a brief summary of the development of these methods. We refer the reader to the works cited for more details.
4. If the Toeplitz matrix T has a singular or ill-conditioned principal submatrix, then a breakdown or near-breakdown can occur in the direct Toeplitz solvers. Such breakdowns will cause numerical instability in subsequent steps of the algorithms and result in inaccurately computed solutions. The question of how to avoid breakdowns or near-breakdowns by skipping over singular submatrices

or ill-conditioned submatrices has been studied extensively, and various such algorithms have been proposed. (See Chan and Hansen [CH92].)

5. The fast direct Toeplitz solvers are in general numerically unstable for indefinite systems. Look-ahead methods are numerically stable and, although it may retain the $O(n^2)$ complexity, it requires $O(n^3)$ operations in the worst case.
6. The stability properties of direct methods for symmetric positive definite Toeplitz systems were discussed in Sweet [Swe84], Bunch [Bun85], Cybenko [Cyb87], and Bojanczyk et al. [BBH95].
7. Gohberg et al. [GKO95] have shown how to perform Gaussian elimination in a fast way for matrices having special displacement structures. Such matrices include Toeplitz, Vandermonde, Hankel, and Cauchy matrices. They have shown how to incorporate partial pivoting into Cauchy solvers. They pointed out that although pivoting cannot be incorporated directly for Toeplitz matrices, Toeplitz problems can be transformed by simple orthogonal operations to Cauchy problems. The solutions to the original problems can be recovered from those of the transformed systems by the reverse orthogonal operations. Thus, fast Gaussian elimination with partial pivoting can be carried out for Toeplitz systems.

Brent and Sweet [BS95] gave a rounding error analysis on the Cauchy and Toeplitz variants of the recent method of Gohberg et al. [GKO95]. It has been shown that the error growth depends on the growth in certain auxiliary vectors, the generators, which are computed by the Gohberg algorithm. In certain circumstances, growth in the generators can be large, so the error growth is much larger than would be encountered when using normal Gaussian elimination with partial pivoting.

48.3 Iterative Toeplitz Solvers

A circulant matrix is a special form of Toeplitz matrix where each row of the matrix is a circular shift of its preceding row. Because of the periodicity, circulant systems can be solved quickly via a deconvolution by discrete Fast Fourier Transforms (FFTs) [Ng04]. Circulant approximations to Toeplitz matrices have been used for some time in signal and image processing [Ng04]. However, in these applications, the circulant approximation so obtained is used to replace the given Toeplitz matrix in subsequent computations. In effect, the matrix equation is changed and, hence, so is the solution.

Development of solely circulant-based iterative methods for Toeplitz systems started in the 1970s. Rino [Rin70] developed a method for generating a series expansion solution to Toeplitz systems by writing a Toeplitz matrix as the sum of a circulant matrix and another Toeplitz matrix and presented a method for choosing the circulant matrix. Silverman and Pearson [SP73] applied similar methods to deconvolution.

In 1986, Strang [Str86] and Olkin [Olk86] independently proposed to precondition Toeplitz matrices by circulant matrices in conjugate gradient iterations. Their motivation was to exploit the fast inversion of circulant matrices. Numerical results in [SE87] and [Olk86] show that the method converges very fast for a wide range of Toeplitz matrices. This has later been proved theoretically in [CS89] and in other papers for other circulant preconditioners [Ng04]. Circulant approximations are used here only as preconditioners for Toeplitz systems and the solutions to the Toeplitz systems are unchanged.

One of the main important results of this methodology is that the complexity of solving a large class of $n \times n$ Toeplitz systems can be reduced to $O(n \log n)$ operations, provided that a suitable preconditioner is used. Besides the reduction of the arithmetic complexity, there are important types of Toeplitz matrix where the fast direct Toeplitz solvers are notoriously unstable, e.g., indefinite and certain non-Hermitian Toeplitz matrices. Therefore, iterative methods provide alternatives for solving these Toeplitz systems.

48.4 Linear Systems with Matrix Structure

This section provides some examples of the latest developments on iterative methods for the iterative solution of linear systems of equations with structured coefficient matrices such as Toeplitz-like, Toeplitz-plus-Hankel, and Toeplitz-plus-band matrices. We would like to make use of their structure to construct some good preconditioners for such matrices.

Facts:

1. Toeplitz-like systems: Let A be an $n \times n$ structured matrix with respect to Z_n (the lower shift matrix):

$$\nabla A_n = A_n - Z_n A_n Z_n^* = G S G^*$$

for some $n \times r$ generator matrix G and $r \times r$ signature matrix $S = (I_p \oplus -I_q)$. If we partition the columns of G into two sets $\{x_i\}_{i=0}^{p-1}$ and $\{y_i\}_{i=0}^{q-1}$,

$$G = [x_0 \ x_1 \ \dots \ x_{p-1} \ y_0 \ y_1 \ \dots \ y_{q-1}] \quad \text{with } p + q = r,$$

then we know from the representation that we can express A as a linear combination of lower triangular Toeplitz matrices,

$$A = \sum_{i=0}^{p-1} L(x_i) L^*(x_i) - \sum_{i=0}^{q-1} L(y_i) L^*(y_i).$$

For example, if $T_{m,n}$ is an $m \times n$ Toeplitz matrix with $m \geq n$, then $T_{m,n}^* T_{m,n}$ is in general not a Toeplitz matrix. However, $T_{m,n}^* T_{m,n}$ does have a small displacement rank, $r \leq 4$, and the displacement representation of $T_{m,n}^* T_{m,n}$ is

$$T_{m,n}^* T_{m,n} = L_n(x_0) L_n(x_0)^* - L_n(y_0) L_n(y_0)^* + L_n(x_1) L_n(x_1)^* - L_n(y_1) L_n(y_1)^*,$$

where

$$x_0 = T_{m,n}^* T_{m,n} e_1 / \|T_{m,n} e_1\|, \quad y_0 = Z_n Z_n^* x_0,$$

$$x_1 = [0, t_{-1}, t_{-2}, \dots, t_{1-n}]^*, \quad \text{and} \quad y_1 = [0, t_{m-1}, t_{m-2}, \dots, t_{m-n+1}]^*.$$

2. For structured matrices with displacement representations, it was suggested in [CNP94] to define the displacement preconditioner to be the circulant approximation of the factors in the displacement representation of A , i.e., the circulant approximation C of A is

$$\sum_{i=0}^{p-1} C(L(x_i)) C^*(L_n(x_i)) - \sum_{i=0}^{q-1} C(L_n(y_i)) C^*(L_n(y_i)).$$

Here, $C(X)$ denotes some circulant approximations to X .

3. The displacement preconditioner approach is applied to Toeplitz least squares and Toeplitz-plus-Hankel least squares problems [Ng04].
4. The systems of linear equations with Toeplitz-plus-Hankel coefficient matrices arise in many signal processing applications. For example, the inverse scattering problem can be formulated as Toeplitz-plus-Hankel systems of equations (see [Ng04].)

The product of P_n and H and the product of H and P_n both give Toeplitz matrices.

Premultiplying P_n to a vector v corresponds to reversing the order of the elements in v . Since

$$Hv = H P_n P_n v$$

and $H P_n$ is a Toeplitz matrix, the Hankel matrix-vector products Hv can be done efficiently using FFTs. A Toeplitz-plus-Hankel matrix can be expressed as $T + H = T + P_n P_n H$.

5. Given circulant preconditioners $C^{(1)}$ and $C^{(2)}$ for Toeplitz matrices T and $P_n H$, respectively, it was proposed in [KK93] to use

$$M = C^{(1)} + P_n C^{(2)}$$

as a preconditioner for the Toeplitz-plus-Hankel matrix $T + H$. With the equality $P_n^2 = I$, we have

$$Mz = C^{(1)}z + P_n C^{(2)} P_n P_n z = v,$$

which is equivalent to

$$P_n Mz = P_n C^{(1)} P_n P_n z + C^{(2)} z = P_n v.$$

By using these two equations, the solution of $z = M^{-1}v$ can be determined.

6. We consider the solution of systems of the form $(T + B)x = b$, where T is a Toeplitz matrix and B is a banded matrix with bandwidth $2b + 1$ independent of the size of the matrix. These systems appear in solving Fredholm integro-differential equations of the form

$$L\{x(\theta)\} + \int_{\alpha}^{\beta} K(\phi - \theta)x(\phi)d\phi = b(\theta).$$

Here, $x(\theta)$ is the unknown function to be found, $K(\theta)$ is a convolution kernel, L is a differential operator and $b(\theta)$ is a given function. After discretization, K will lead to a Toeplitz matrix, L to a banded matrix, and $b(\theta)$ to the right-hand side vector [Ng04]. Toeplitz-plus-band matrices also appear in signal processing literature and have been referred to as perihel innovation matrices.

Unlike Toeplitz systems, there exist no fast direct solvers for solving Toeplitz-plus-band systems. It is mainly because the displacement rank of the matrix $T + B$ can take any value between 0 and n . Hence, fast Toeplitz solvers that are based on small displacement rank of the matrices cannot be applied. Conjugate gradient methods with circulant preconditioners do not work for Toeplitz-plus-band systems either. The main reason is that when the eigenvalues of B are not clustered, the matrix $C(T) + B$ cannot be inverted easily.

In [CN93], it was proposed to use the matrix $E + B$ to precondition $T + B$, where E is the band-Toeplitz preconditioner such that E is spectrally equivalent to T . Note that E is a banded matrix, and the banded system $(E + B)y = z$ can be solved by using any band matrix solver.

7. Banded preconditioners are successfully applied to precondition Sinc–Galerkin systems (Toeplitz-plus-band systems) arising from the Sinc–Galerkin method to partial differential equations (see [Ng04].)
8. In most of applications we simply use the circulant or other transform-based preconditioners [Ng04]. We can extend the results for point circulant or point transform-based preconditioners to block circulant preconditioners or block transform-based preconditioners [Jin02] for block-Toeplitz, Toeplitz-block, and Toeplitz-block–block-Toeplitz matrices.
9. Consider the system $(A \otimes B)x = b$, where A is an m -by- m Hermitian positive definite matrix and B is an n -by- n Hermitian positive definite Toeplitz matrix. By using a circulant approximation $C(B)$ to B , the preconditioned system becomes

$$(A \otimes C(B))^{-1}(A \otimes B)x = (A \otimes C(B))^{-1}b,$$

or

$$(I \otimes C(B)^{-1}B)x = (A^{-1} \otimes C(B)^{-1})b.$$

When B is a Hermitian positive definite Toeplitz matrix, $C(B)$ can be obtained in $O(n)$ operation [Jin02], [Ng04]. The initialization cost is about $O(m^3 + m^2n + mn \log n)$ operations. Moreover, since the cost of multiplying B by becomes $O(n \log n)$, we see that the cost per iteration is equal to $O(mn \log n)$ when iterative methods are employed.

10. When Toeplitz matrices have full rank, Toeplitz least squares problems

$$\min \|Tx - b\|_2^2$$

are equivalent to solving the normal equation matrices

$$T^* T x = T^* b.$$

Circulant preconditioners can be applied effectively and efficiently to solving Toeplitz least squares problems if Toeplitz matrices have full rank. When Toeplitz matrices do not have full rank, it is still an open research problem to find efficient algorithms for solving rank-deficient Toeplitz least squares problem. One possibility is to consider the generalized inverses of Toeplitz matrices. In the literature, computing the inverses and the generalized inverses of structured matrices are important practical computational problems. (See, for instance, Pan and Rami [PR01] and Bini et al. [BCV03].)

11. Instead of Toeplitz least squares problems $\min \|Tx - b\|_2^2$, we are interested in the 1-norm problem, i.e., $\min \|Tx - b\|_1$. The advantage of using the 1-norm is that the solution is more robust than using the 2-norm in statistical estimation problems. In particular, a small number of outliers have less influence on the solution. It is interesting to develop efficient algorithms for solving 1-norm Toeplitz least squares problems. Fu et al. [FNN06] have considered the least absolute deviation (LAD) solution of image restoration problems.
12. It is interesting to find good preconditioners for Toeplitz-related systems with large displacement rank. Good examples are Toeplitz-plus-band systems studied. Direct Toeplitz-like solvers cannot be employed because of the large displacement rank. However, iterative methods are attractive since coefficient matrix–vector products can be computed efficiently at each iteration. For instance, for the Toeplitz-plus-band matrix, its matrix-vector product can be computed in $O(n \log n)$ operations. The main concern is how to design good preconditioners for such Toeplitz-related systems with large displacement rank. Recently, Lin et al. [LNC05] proposed and developed factorized banded inverse preconditioners for matrices with Toeplitz structure. Also, Lin et al. [LCN04] studied incomplete factorization-based preconditioners for Toeplitz-like systems with large displacement ranks in image processing.

48.5 Total Least Squares Problems

1. The least squares problem $Tf \approx g$ is

$$\min_f \|Tf - g\|_2.$$

If the matrix T is known exactly, but the vector g is corrupted by random errors that are uncorrelated with zero mean and equal variance, then the least squares solution provides the best unbiased estimate of f . However, if T is also corrupted by errors, then the total least squares (TLS) method may be more appropriate. The TLS problem minimizes

$$\min_{\hat{T}, \hat{g}} \|[T \ g] - [\hat{T} \ \hat{g}]\|_F^2$$

with the constraint $\hat{T} \hat{f} = \hat{g}$. If the smallest singular value of T is larger than the smallest singular value σ^2 of $[T \ g]$, then there exists a unique TLS solution f_{TLS} , which can be represented as the solution to the normal equations:

$$(T^T T - \sigma^2 I) f = T^T g,$$

or as the solution to the eigenvalue problem:

$$\begin{bmatrix} T^T T & T^T g \\ g^T T & g^T g \end{bmatrix} \begin{bmatrix} f \\ -1 \end{bmatrix} = \sigma^2 \begin{bmatrix} f \\ -1 \end{bmatrix}.$$

Kamm and Nagy [KN98] proposed using Newton and Rayleigh quotient iterations for large TLS Toeplitz problems. Their method is a modification of a method suggested by Cybenko and Van Loan [CV86] for computing the minimum eigenvalue of a symmetric positive definite Toeplitz matrix. Specifically, first note that the TLS solution f_{TLS} solves the eigenvalue problem. Moreover, this eigenvalue problem is equivalent to

$$T^T T f - T^T g = \sigma^2 f$$

and

$$g^T T f - g^T g = -\sigma^2,$$

which can be combined to obtain the following secular equation for σ^2 :

$$g^T g - g^T T (T^T T - \sigma^2 I)^{-1} T^T g - \sigma^2 = 0.$$

Therefore, σ^2 is the smallest root of the rational equation

$$h(\sigma^2) = g^T g - g^T T (T^T T - \sigma^2 I)^{-1} T^T g - \sigma^2$$

and can be found using Newton's method. Note that if σ^2 is less than the smallest singular value δ^2 of T , then the matrix $T^T T - \sigma^2 I$ is positive definite. Assume for now that the initial estimate is within the interval $[\sigma^2, \delta^2]$. An analysis given in [CV86] shows that subsequent Newton iterates will remain within this interval and will converge from the right to σ^2 .

2. In the above computation, $\hat{T}$ is not necessary to have Toeplitz structure. In another development, Ng [NPP00] presented an iterative, regularized, and constrained total least squares algorithm by requiring $\hat{T}$ to be Toeplitz. Preliminary numerical tests are reported on some simulated optical imaging problems. The numerical results showed that the regularized constrained TLS method is better than the regularized least squares method.
3. Other interesting areas are to design efficient algorithms based on preconditioning techniques for finding eigenvalues and singular values of Toeplitz-like matrices. Ng [Ng00] has employed preconditioned Lanczos methods for the minimum eigenvalue of a symmetric positive definite Toeplitz matrix.

References

- [AG88] G. Ammar and W. Gragg, Superfast solution of real positive definite Toeplitz systems, *SIAM J. Matrix Anal. Appl.*, 9:61–67, 1988.
- [BTY01] D. Bini, E. Tyrtyshnikov, and P. Yalamov, *Structured Matrices: Recent Developments in Theory and Computation*, Nova Science Publishers, Inc., New York, 2001.
- [BCV03] D. Bini, G. Codevico, and M. Van Barel, Solving Toeplitz least squares problems by means of Newton's iterations, *Num. Algor.*, 33:63–103, 2003.
- [BA80] R. Bitmead and B. Anderson, Asymptotically fast solution of Toeplitz and related systems of linear equations, *Lin. Alg. Appl.*, 34:103–116, 1980.
- [BBH95] A. Bojanczyk, R. Brent, F. de Hoog, and D. Sweet, On the stability of the Bareiss and related Toeplitz factorization algorithms, *SIAM J. Matrix Anal. Appl.*, 16:40–57, 1995.
- [BGY80] R. Brent, F. Gustavson, and D. Yun, Fast solution of Toeplitz systems of equations and computation of Padé approximants, *J. Algor.*, 1:259–295, 1980.
- [BS95] R. Brent and D. Sweet, Error analysis of a partial pivoting method for structured matrices, Technical report, The Australian National University, Canberra, 1995.
- [Bun85] J. Bunch, Stability of methods for solving Toeplitz systems of equations, *SIAM J. Sci. Stat. Comp.*, 6:349–364, 1985.
- [CNP94] R. Chan, J. Nagy, and R. Plemmons, Displacement preconditioner for Toeplitz least squares iterations, *Elec. Trans. Numer. Anal.*, 2:44–56, 1994.

- [CN93] R. Chan and M. Ng, Fast iterative solvers for Toeplitz-plus-band systems, *SIAM J. Sci. Comput.*, 14:1013–1019, 1993.
- [CS89] R. Chan and G. Strang, Toeplitz equations by conjugate gradients with circulant preconditioner, *SIAM J. Sci. Stat. Comput.*, 10:104–119, 1989.
- [CH92] T. Chan and P. Hansen, A look-ahead Levinson algorithm for general Toeplitz systems, *IEEE Trans. Signal Process.*, 40:1079–1090, 1992.
- [Cyb87] G. Cybenko, Fast Toeplitz orthogonalization using inner products, *SIAM J. Sci. Stat. Comput.*, 8:734–740, 1987.
- [CV86] G. Cybenko and C. Van Loan, Computing the minimum eigenvalue of a symmetric positive definite Toeplitz matrix, *SIAM J. Sci. Stat. Comput.*, 7:123–131, 1986.
- [Dur60] J. Durbin, The fitting of time series models, *Rev. Int. Stat.*, 28:233–244, 1960.
- [FNN06] H. Fu, M. Ng, M. Nikolova, and J. Barlow, Efficient minimization methods of mixed l2-l1 and l1-l1 norms for image restoration, *SIAM J. Sci. Comput.*, 27:1881–1902, 2006.
- [GKO95] I. Gohberg, T. Kailath, and V. Olshevsky, Fast Gaussian elimination with partial pivoting for matrices with displacement structure, *Math. Comp.*, 64:65–72, 1995.
- [GS84] U. Grenander and G. Szegö, *Toeplitz Forms and Their Applications*, Chelsea Publishing, 2nd ed., New York, 1984.
- [Hoo87] F. de Hoog, A new algorithm for solving Toeplitz systems of equations, *Lin. Alg. Appl.*, 88/89:123–138, 1987.
- [Jin02] X. Jin, *Developments and Applications of Block Toeplitz Iterative Solvers*, Kluwer Academic Publishers, New York, 2002.
- [KS98] T. Kailath and A. Sayed, *Fast Reliable Algorithms for Matrices with Structured*, SIAM, Philadelphia, 1998.
- [KN98] J. Kamm and J. Nagy, A total least squares method for Toeplitz systems of equations, *BIT*, 38:560–582, 1998.
- [KK93] T. Ku and C. Kuo, Preconditioned iterative methods for solving Toeplitz-plus-Hankel systems, *SIAM J. Numer. Anal.*, 30:824–845, 1993.
- [Lev46] N. Levinson, The Wiener rms (root mean square) error criterion in filter design and prediction, *J. Math. Phys.*, 25:261–278, 1946.
- [LCN04] F. Lin, W. Ching, and M. Ng, Preconditioning regularized least squares problems arising from high-resolution image reconstruction from low-resolution frames, *Lin. Alg. Appl.*, 301:149–168, 2004.
- [LNC05] F. Lin, M. Ng, and W. Ching, Factorized banded inverse preconditioners for matrices with Toeplitz structure, *SIAM J. Sci. Comput.*, 26:1852–1870, 2005.
- [Mor80] M. Morf, Doubling algorithms for Toeplitz and related equations, in *Proc. IEEE Internat. Conf. on Acoustics, Speech and Signal Processing*, pp. 954–959, Denver, CO, 1980.
- [Ng00] M. Ng, Preconditioned Lanczos methods for the minimum eigenvalue of a symmetric positive definite Toeplitz matrix, *SIAM J. Sci. Comput.*, 21:1973–1986, 2000.
- [Ng04] M. Ng, *Iterative Methods for Toeplitz Systems*, Oxford University Press, UK, 2004.
- [NPP00] M. Ng, R. Plemmons, and F. Pimentel, A new approach to constrained total least squares image restoration, *Lin. Alg. Appl.*, 316: 237–258, 2000.
- [Olk86] J. Olkin, Linear and nonlinear deconvolution problems, Technical report, Rice University, Houston, TX, 1986.
- [PR01] V. Pan and Y. Rami, Newton’s iteration for the inversion of structured matrices, in D. Bini, E. Tyrtyshnikov, and P. Yalov, Eds., *Structured Matrices: Recent Developments in Theory and Computation*, pp. 79–90, Nova Science Pub., Hauppauge, NY, 2001.
- [Rin70] C. Rino, The inversion of covariance matrices by finite Fourier transforms, *IEEE Trans. Inform. Theory*, 16:230–232, 1970.
- [Sch17] I. Schur, Über potenzreihen, die im innern des einheitskreises beschränkt sind, *J. Reine Angew. Math.*, 147:205–232, 1917.

- [SP73] H. Silverman and A. Pearson, On deconvolution using the discrete Fourier transform, *IEEE Trans. Audio Electroacous.*, 21:112–118, 1973.
- [Str86] G. Strang, A proposal for Toeplitz matrix calculations, *Stud. Appl. Math.*, 74:171–176, 1986.
- [SE87] G. Strang and A. Edelman, The Toeplitz-circulant eigenvalue problem $ax = \lambda cx$, in L. Bragg and J. Dettman, Eds., *Oakland Conf. on PDE and Appl. Math.*, New York, Longman Sci. Tech., 1987.
- [Swe84] D. Sweet, Fast Toeplitz orthogonalization, *Numer. Math.*, 43:1–21, 1984.
- [Tre64] W. Trench, An algorithm for the inversion of finite Toeplitz matrices, *SIAM J. Appl. Math.*, 12:515–522, 1964.

49

Large-Scale Matrix Computations

49.1	Basic Concepts	49-1
49.2	Sparse Matrix Factorizations	49-3
49.3	Krylov Subspaces	49-5
49.4	The Symmetric Lanczos Process	49-6
49.5	The Nonsymmetric Lanczos Process	49-8
49.6	The Arnoldi Process	49-10
49.7	Eigenvalue Computations	49-12
49.8	Linear Systems of Equations	49-12
49.9	Dimension Reduction of Linear Dynamical Systems.....	49-14
	References	49-16

Roland W. Freund

University of California, Davis

Computational problems, especially in science and engineering, often involve large matrices. Examples of such problems include large sparse systems of linear equations [FGN92],[Saa03],[vdV03], e.g., arising from discretizations of partial differential equations, eigenvalue problems for large matrices [BDD00], [LM05], linear time-invariant dynamical systems with large state-space dimensions [FF94],[FF95],[Fre03], and large-scale linear and nonlinear optimization problems [KR91],[Wri97],[NW99],[GMS05]. The large matrices in these problems exhibit special structures, such as sparsity, that can be exploited in computational procedures for their solution. Roughly speaking, computational problems involving matrices are called “large-scale” if they can be solved only by methods that exploit these special matrix structures.

In this section, as in Chapter 44, multiplication of a vector $\mathbf{v}$ by a scalar λ is denoted by $\mathbf{v}\lambda$ rather than $\lambda\mathbf{v}$.

49.1 Basic Concepts

Many of the most efficient algorithms for large-scale matrix computations are based on approximations of the given large matrix by small matrices obtained via Petrov–Galerkin projections onto suitably chosen small-dimensional subspaces. In this section, we present some basic concepts of such projections.

Definitions:

Let $C \in \mathbb{C}^{n \times n}$ and let $V_j = [\mathbf{v}_1 \quad \mathbf{v}_2 \quad \cdots \quad \mathbf{v}_j] \in \mathbb{C}^{n \times j}$ be a matrix with orthonormal columns, i.e.,

$$\mathbf{v}_i^* \mathbf{v}_k = \begin{cases} 0 & \text{if } i \neq k, \\ 1 & \text{if } i = k, \end{cases} \quad \text{for all } i, k = 1, 2, \dots, j.$$

The matrix

$$C_j := V_j^* C V_j \in \mathbb{C}^{j \times j}$$

is called the **orthogonal Petrov–Galerkin projection** of C onto the subspace

$$S = \text{span}\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_j\}$$

of $\mathbb{C}^n$ spanned by the columns of V_j .

Let $C \in \mathbb{C}^{n \times n}$, and let $V_j = [\mathbf{v}_1 \ \mathbf{v}_2 \ \dots \ \mathbf{v}_j] \in \mathbb{C}^{n \times j}$ and $W_j = [\mathbf{w}_1 \ \mathbf{w}_2 \ \dots \ \mathbf{w}_j] \in \mathbb{C}^{n \times j}$ be two matrices such that $W_j^T V_j$ is nonsingular. The matrix

$$C_j := (W_j^T V_j)^{-1} W_j^T C V_j \in \mathbb{C}^{j \times j}$$

is called the **oblique Petrov–Galerkin projection** of C onto the subspace

$$S = \text{span}\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_j\}$$

of $\mathbb{C}^n$ spanned by the columns of V_j and orthogonally to the subspace

$$T = \text{span}\{\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_j\}$$

of $\mathbb{C}^n$ spanned by the columns of W_j .

A **flop** is the work associated with carrying out any one of the elementary operations $a + b$, $a - b$, ab , or a/b , where $a, b \in \mathbb{C}$, in floating-point arithmetic.

Let $A = [a_{ik}] \in \mathbb{C}^{m \times n}$ be a given matrix. Matrix-vector multiplications with A are said to be **fast** if for any $\mathbf{x} \in \mathbb{C}^n$, the computation of $\mathbf{y} = A\mathbf{x}$ requires significantly fewer than $2mn$ flops.

A matrix $A = [a_{ik}] \in \mathbb{C}^{m \times n}$ is said to be **sparse** if only a small fraction of its entries a_{ik} are nonzero.

For a sparse matrix $A = [a_{ik}] \in \mathbb{C}^{m \times n}$, $\text{nnz}(A)$ denotes the number of nonzero entries of A .

A matrix $A = [a_{ik}] \in \mathbb{C}^{m \times n}$ is said to be **dense** if most of its entries a_{ik} are nonzero.

Facts:

The following facts on sparse matrices can be found in [Saa03, Chap. 3] and the facts on computing Petrov–Galerkin projections of matrices in [Saa03, Chap. 6].

1. For a sparse matrix $A = [a_{ik}] \in \mathbb{C}^{m \times n}$, only its nonzero or potentially nonzero entries a_{ik} , together with their row and column indices i and k , need to be stored.
2. Matrix-vector multiplications with a sparse matrix $A = [a_{ik}] \in \mathbb{C}^{m \times n}$ are fast. More precisely, for any $\mathbf{x} \in \mathbb{C}^n$, $\mathbf{y} = A\mathbf{x}$ can be computed with at most $2\text{nnz}(A)$ flops.
3. If $C \in \mathbb{C}^{n \times n}$ and $j \ll n$, the computational cost for computing the orthogonal Petrov–Galerkin projection of C onto the j -dimensional subspace $S = \text{span}\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_j\}$ of $\mathbb{C}^n$ is dominated by the j matrix-vector products $\mathbf{y}_i = C\mathbf{v}_i$, $i = 1, 2, \dots, j$.
4. If $C \in \mathbb{C}^{n \times n}$ and $j \ll n$, the computational cost for computing the oblique Petrov–Galerkin projection of C onto the j -dimensional subspace $S = \text{span}\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_j\}$ of $\mathbb{C}^n$ and orthogonally to the j -dimensional subspace $T = \text{span}\{\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_j\}$ of $\mathbb{C}^n$ is dominated by the j matrix-vector products $\mathbf{y}_i = C\mathbf{v}_i$, $i = 1, 2, \dots, j$.
5. If matrix-vector products with a large matrix $C \in \mathbb{C}^{n \times n}$ are fast, then orthogonal and oblique Petrov–Galerkin projections C_j of C can be generated with low computational cost.

49.2 Sparse Matrix Factorizations

In this section, we present some basic concepts of sparse matrix factorizations. A more detailed description can be found in [DER89].

Definitions:

Let $A \in \mathbb{C}^{n \times n}$ be a sparse nonsingular matrix. A **sparse LU factorization** of A is a factorization of the form

$$A = PLUQ,$$

where $P, Q \in \mathbb{R}^{n \times n}$ are permutation matrices, $L \in \mathbb{C}^{n \times n}$ is a sparse unit lower triangular matrix, and $U \in \mathbb{C}^{n \times n}$ is a sparse nonsingular upper triangular matrix.

Fill-in of a sparse LU factorization $A = PLUQ$ is the set of nonzero entries of L and U that appear in positions (i, k) where $a_{ik} = 0$.

Let $A = A^* \in \mathbb{C}^{n \times n}$, $A \succ 0$, be a sparse Hermitian positive definite matrix. A **sparse Cholesky factorization** of A is a factorization of the form

$$A = PLL^*P^T,$$

where $P \in \mathbb{R}^{n \times n}$ is a permutation matrix and $L \in \mathbb{C}^{n \times n}$ is a sparse lower triangular matrix.

Fill-in of a sparse Cholesky factorization $A = PLL^*P^T$ is the set of nonzero entries of L that appear in positions (i, k) where $a_{ik} = 0$.

Let $T \in \mathbb{C}^{n \times n}$ be a sparse nonsingular (upper or lower) triangular matrix, and let $\mathbf{b} \in \mathbb{C}^n$. A **sparse triangular solve** is the solution of a linear system

$$T\mathbf{x} = \mathbf{b}$$

with a sparse triangular coefficient matrix T .

Facts:

The following facts can be found in [DER89].

1. The permutation matrices P and Q in a sparse LU factorization of A allow for reorderings of the rows and columns of A . These reorderings serve two purposes. First, they allow for pivoting for numerical stability in order to avoid division by the number 0 or by numbers close to 0, which would result in breakdowns or numerical instabilities in the procedure used for the computation of the factorization. Second, the reorderings allow for pivoting for sparsity, the goal of which is to minimize the amount of fill-in.
2. For Cholesky factorizations of matrices $A = A^* \succ 0$, the positive definiteness of A implies that pivoting for numerical stability is not needed. Therefore, the permutation matrix P in a sparse Cholesky factorization serves the single purpose of pivoting for sparsity.
3. For both sparse LU and sparse Cholesky factorizations, the problem of “optimal” pivoting for sparsity, i.e., finding reorderings that minimize the amount of fill-in, is NP-complete. This means that for practical purposes, minimizing the amount of fill-in of factorizations of large sparse matrices is impossible in general. However, there are a large number of pivoting strategies that — while not minimizing fill-in — efficiently limit the amount of fill-in for many important classes of large sparse matrices. (See, e.g., [DER89].)
4. A sparse triangular solve with the matrix T requires at most $2nnz(T)$ flops.
5. Not every large sparse matrix A has a sparse LU factorization with limited amounts of fill-in. For example, LU or Cholesky factorizations of sparse matrices A arising from discretization of partial differential equations for three-dimensional problems are often prohibitive due to the large amount of fill-in.

Examples:

1. Given a sparse LU factorization $A = PLUQ$ of a sparse nonsingular matrix $A \in \mathbb{C}^{n \times n}$, the solution $\mathbf{x}$ of the linear system $A\mathbf{x} = \mathbf{b}$ with any right-hand side $\mathbf{b} \in \mathbb{C}^n$ can be computed as follows:

$$\begin{aligned} \text{Set } \mathbf{c} &= P^T \mathbf{b}, \\ \text{Solve } L\mathbf{z} &= \mathbf{c} \text{ for } \mathbf{z}, \\ \text{Solve } U\mathbf{y} &= \mathbf{z} \text{ for } \mathbf{y}, \\ \text{Set } \mathbf{x} &= Q^T \mathbf{y}. \end{aligned}$$

Since P and Q are permutation matrices, the first and the last steps are just reorderings of the entries of the vectors $\mathbf{b}$ and $\mathbf{y}$, respectively. Therefore, the main computational cost is the two triangular solves with L and U , which requires at most $2(\text{nnz}(L) + \text{nnz}(U))$ flops.

2. Given a sparse Cholesky factorization $A = PLL^*P^T$ of a sparse Hermitian positive definite matrix $A \in \mathbb{C}^{n \times n}$, the solution $\mathbf{x}$ of the linear system $A\mathbf{x} = \mathbf{b}$ with any right-hand side $\mathbf{b} \in \mathbb{C}^n$ can be computed as follows:

$$\begin{aligned} \text{Set } \mathbf{c} &= P^T \mathbf{b}, \\ \text{Solve } L\mathbf{z} &= \mathbf{c} \text{ for } \mathbf{z}, \\ \text{Solve } L^*\mathbf{y} &= \mathbf{z} \text{ for } \mathbf{y}, \\ \text{Set } \mathbf{x} &= P^T \mathbf{y}. \end{aligned}$$

Since P is a permutation matrix, the first and the last steps are just reorderings of the entries of the vectors $\mathbf{b}$ and $\mathbf{y}$, respectively. Therefore, the main computational cost is the two triangular solves with L and L^* , which requires at most $4\text{nnz}(L)$ flops.

3. In large-scale matrix computations, sparse factorizations are often not applied to a given sparse matrix $A \in \mathbb{C}^{n \times n}$, but to a suitable “approximation” $A_0 \in \mathbb{C}^{n \times n}$ of A . For example, if sparse factorizations of A itself are prohibitive due to excessive fill-in, such approximations A_0 can often be obtained by computing an “incomplete” factorization of A that simply discards unwanted fill-in entries. Given a sparse LU factorization

$$A_0 = PLUQ$$

of a sparse nonsingular matrix $A_0 \in \mathbb{C}^{n \times n}$, which in some sense approximates the original matrix $A \in \mathbb{C}^{n \times n}$, one then uses iterative procedures that only involve matrix-vector products with the matrix

$$C := A_0^{-1}A = Q^T U^{-1} L^{-1} P^T A,$$

or possibly its transpose C^T . In the context of solving linear systems $A\mathbf{x} = \mathbf{b}$, the matrix A_0 is called a **preconditioner**, and the matrix C is called the **preconditioned** coefficient matrix.

In general, the matrix $C = A_0^{-1}A$ is full. However, if C is only used in the form of matrix-vector products, then there is no need to explicitly form C . Instead, for any $\mathbf{v} \in \mathbb{C}^n$, the result of the matrix-vector product $\mathbf{y} = C\mathbf{v}$ can be computed as follows:

$$\begin{aligned} \text{Set } \mathbf{c} &= A\mathbf{v}, \\ \text{Set } \mathbf{d} &= P^T \mathbf{c}, \\ \text{Solve } L\mathbf{f} &= \mathbf{d} \text{ for } \mathbf{f}, \\ \text{Solve } U\mathbf{z} &= \mathbf{f} \text{ for } \mathbf{z}, \\ \text{Set } \mathbf{y} &= Q^T \mathbf{z}. \end{aligned}$$

Since P and Q are permutation matrices, the second and the last steps are just reorderings of the entries of the vectors $\mathbf{c}$ and $\mathbf{z}$, respectively. Therefore, the main computational cost is the matrix-vector product with the sparse matrix A in the first step, the triangular solve with L in the

third step, and the triangular solve with U in the fourth step, which requires a total of at most $2(\text{nnz}(A) + \text{nnz}(L) + \text{nnz}(U))$ flops. Similarly, each matrix product with C^T can be computed with at most $2(\text{nnz}(A) + \text{nnz}(L) + \text{nnz}(U))$ flops. In particular, matrix-vector products with both C and C^T are fast.

4. For sparse Hermitian matrices $A = A^* \in \mathbb{C}^{n \times n}$, preconditioning is often applied in a symmetric manner. Suppose

$$A_0 = PLL^*P^T$$

is a sparse Cholesky factorization of a sparse matrix $A_0 = A_0^* \in \mathbb{C}^{n \times n}$, $A_0 \succ 0$, which in some sense approximates the original matrix A . Then the symmetrically preconditioned matrix C is defined as

$$C := (PL)^{-1}A(L^*P^T)^{-1} = L^{-1}P^TAP(L^*)^{-1}.$$

Note that $C = C^*$ is a Hermitian matrix. For any $\mathbf{v} \in \mathbb{C}^n$, the result of the matrix-vector product $\mathbf{y} = C\mathbf{v}$ can be computed as follows:

$$\begin{aligned} \text{Solve } L^*\mathbf{c} &= \mathbf{v} \quad \text{for } \mathbf{c}, \\ \text{Set } \mathbf{d} &= P\mathbf{c}, \\ \text{Set } \mathbf{f} &= A\mathbf{d}, \\ \text{Set } \mathbf{z} &= P^T\mathbf{f}, \\ \text{Solve } L\mathbf{y} &= \mathbf{z} \quad \text{for } \mathbf{y}. \end{aligned}$$

The main computational cost is the triangular solve with L^* in the first step, the matrix-vector product with the sparse matrix A in the third step, and the triangular solve with L in the last step, which requires a total of at most $2(\text{nnz}(A) + 2\text{nnz}(L))$ flops. In particular, matrix-vector products with C are fast.

49.3 Krylov Subspaces

Petrov–Galerkin projections are often used in conjunction with Krylov subspaces. In this section, we present the basic concepts of Krylov subspaces. In the following, it is assumed that $C \in \mathbb{C}^{n \times n}$ and $\mathbf{r} \in \mathbb{C}^n$, $\mathbf{r} \neq \mathbf{0}$.

Definitions:

The sequence

$$\mathbf{r}, C\mathbf{r}, C^2\mathbf{r}, \dots, C^{j-1}\mathbf{r}, \dots$$

is called the **Krylov sequence** induced by C and $\mathbf{r}$.

Let $j \geq 1$. The subspace

$$K_j(C, \mathbf{r}) := \text{span}\{\mathbf{r}, C\mathbf{r}, C^2\mathbf{r}, \dots, C^{j-1}\mathbf{r}\}$$

of $\mathbb{C}^n$ spanned by the first j vectors of the Krylov sequence is called the **j th Krylov subspace** induced by C and $\mathbf{r}$.

A sequence of linearly independent vectors

$$\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_j \in \mathbb{C}^n$$

is said to be a **nested basis** for the j th Krylov subspace $K_j(C, \mathbf{r})$ if

$$\text{span}\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_i\} = K_i(C, \mathbf{r}) \quad \text{for all } i = 1, 2, \dots, j.$$

Let $p(\lambda) = c_0 + c_1\lambda + c_2\lambda^2 + \cdots + c_{d-1}\lambda^{d-1} + \lambda^d$ be a monic polynomial of degree d with coefficients in $\mathbb{C}$. The **minimal polynomial of C with respect to $\mathbf{r}$** is the unique monic polynomial of smallest possible degree for which $p(C)\mathbf{r} = \mathbf{0}$.

The **grade of C with respect to $\mathbf{r}$** , $d(C, \mathbf{r})$, is the degree of the minimal polynomial of C and $\mathbf{r}$.

Facts:

The following facts can be found in [Hou75, Sect. 1.5], [SB02, Sect. 6.3], or [Saa03, Sect. 6.2].

1. The vectors

$$\mathbf{r}, C\mathbf{r}, C^2\mathbf{r}, \dots, C^{j-1}\mathbf{r}$$

are linearly independent if and only if $j \leq d(C, \mathbf{r})$.

2. Let $d = d(C, \mathbf{r})$. The vectors

$$\mathbf{r}, C\mathbf{r}, C^2\mathbf{r}, \dots, C^{d-2}\mathbf{r}, C^{d-1}\mathbf{r}, C^j\mathbf{r}$$

are linearly dependent for all $j > d$.

3. The dimension of the j th Krylov subspace $K_j(C, \mathbf{r})$ is given by

$$\dim K_j(C, \mathbf{r}) = \begin{cases} j & \text{if } j \leq d(C, \mathbf{r}), \\ d(C, \mathbf{r}) & \text{if } j > d(C, \mathbf{r}). \end{cases}$$

4. $d(C, \mathbf{r}) = \text{rank} [\mathbf{r} \quad C\mathbf{r} \quad C^2\mathbf{r} \quad \cdots \quad C^{n-1}\mathbf{r}]$.

49.4 The Symmetric Lanczos Process

In this section, we assume that $C = C^* \in \mathbb{C}^{n \times n}$ is a Hermitian matrix and that $\mathbf{r} \in \mathbb{C}^n$, $\mathbf{r} \neq \mathbf{0}$ is a nonzero starting vector. We discuss the **symmetric Lanczos process** [Lan50] for constructing a nested basis for the Krylov subspace $K_j(C, \mathbf{r})$ induced by C and $\mathbf{r}$.

Algorithm (Symmetric Lanczos process)

Compute $\beta_1 = \|\mathbf{r}\|_2$, and set $\mathbf{v}_1 = \mathbf{r}/\beta_1$, and $\mathbf{v}_0 = \mathbf{0}$.

For $j = 1, 2, \dots$, do:

- 1) Compute $\mathbf{v} = C\mathbf{v}_j$, and set $\mathbf{v} = \mathbf{v} - \mathbf{v}_{j-1}\beta_j$.
- 2) Compute $\alpha_j = \mathbf{v}_j^*\mathbf{v}$, and set $\mathbf{v} = \mathbf{v} - \mathbf{v}_j\alpha_j$.
- 3) Compute $\beta_{j+1} = \|\mathbf{v}\|_2$.
If $\beta_{j+1} = 0$, stop.
Otherwise, set $\mathbf{v}_{j+1} = \mathbf{v}/\beta_{j+1}$.

end for

Facts:

The following facts can be found in [CW85], [SB02, Sect. 6.5.3], or [Saa03, Sect. 6.6].

1. In exact arithmetic, the algorithm stops after a finite number of iterations. More precisely, it stops when $j = d(C, \mathbf{r})$ is reached.

2. The Lanczos vectors

$$\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_j$$

generated during the first j iterations of the algorithm form a nested basis for the j th Krylov subspace $K_j(C, \mathbf{r})$.

- The Lanczos vectors satisfy the three-term recurrence relations

$$\mathbf{v}_{i+1}\beta_{i+1} = C\mathbf{v}_i - \mathbf{v}_i\alpha_i - \mathbf{v}_{i-1}\beta_i, \quad i = 1, 2, \dots, j.$$

- These three-term recurrence relations can be written in compact matrix form as follows:

$$CV_j = V_j T_j + \beta_{j+1}\mathbf{v}_{j+1}\mathbf{e}_j^T = V_{j+1}T_j^{(e)}.$$

Here, we set

$$V_j = [\mathbf{v}_1 \quad \mathbf{v}_2 \quad \dots \quad \mathbf{v}_j], \quad \mathbf{e}_j^T = [0 \quad 0 \quad \dots \quad 0 \quad 1] \in \mathbb{R}^{1 \times j},$$

$$T_j = \begin{bmatrix} \alpha_1 & \beta_2 & 0 & \dots & 0 \\ \beta_2 & \alpha_2 & \beta_3 & \ddots & \vdots \\ 0 & \beta_3 & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta_j \\ 0 & \dots & 0 & \beta_j & \alpha_j \end{bmatrix}, \quad T_j^{(e)} = \begin{bmatrix} T_j \\ \beta_{j+1}\mathbf{e}_j^T \end{bmatrix}, \quad \text{and} \quad V_{j+1} = [V_j \quad \mathbf{v}_{j+1}].$$

Note that $T_j \in \mathbb{C}^{j \times j}$ and $T_j^{(e)} \in \mathbb{C}^{(j+1) \times j}$ are tridiagonal matrices.

- In exact arithmetic, the Lanczos vectors are orthonormal. Since the Lanczos vectors are the columns of V_j , this orthonormality can be stated compactly as follows:

$$V_j^* V_j = I_j \quad \text{and} \quad V_j^* \mathbf{v}_{j+1} = \mathbf{0}.$$

- These orthogonality relations, together with the above compact form of the three-term recurrence relations, imply that

$$T_j = V_j^* C V_j.$$

Thus, the j th **Lanczos matrix** T_j is the orthogonal Petrov–Galerkin projection of C onto the j th Krylov subspace $K_j(C, \mathbf{r})$.

- The computational cost of each j th iteration of the symmetric Lanczos process is fixed, and it is dominated by the matrix-vector product $\mathbf{v} = C\mathbf{v}_j$. In particular, the computational cost for generating the orthogonal Petrov–Galerkin projection T_j of C is dominated by the j matrix-vector products with C .
- If C is a sparse matrix or a preconditioned matrix with a sparse preconditioner, then the matrix-vector products with C are fast. In this case, the symmetric Lanczos process is a very efficient procedure for computing orthogonal Petrov–Galerkin projections T_j of C onto Krylov subspaces $K_j(C, \mathbf{r})$.
- The three-term recurrence relations used to generate the Lanczos vectors explicitly enforce orthogonality only among each set of three consecutive vectors, $\mathbf{v}_{j-1}$, $\mathbf{v}_j$, and $\mathbf{v}_{j+1}$. As a consequence, in finite-precision arithmetic, round-off error will usually cause loss of orthogonality among all Lanczos vectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_{j+1}$.
- For applications of the Lanczos process in large-scale matrix computations, this loss of orthogonality is often benign, and only delays convergence. More precisely, in such applications, the Lanczos matrix $T_j \in \mathbb{C}^{j \times j}$ for some $j \ll n$ is used to obtain an approximate solution of a matrix problem involving the large matrix $C \in \mathbb{C}^{n \times n}$. Due to round-off error and the resulting loss of orthogonality, the number j of iterations that is needed to obtain a satisfactory approximate solution is larger than the number of iterations that would be needed in exact arithmetic.

49.5 The Nonsymmetric Lanczos Process

In this section, we assume that $C \in \mathbb{C}^{n \times n}$ is a general square matrix, and that $\mathbf{r} \in \mathbb{C}^n$, $\mathbf{r} \neq \mathbf{0}$, and $\mathbf{l} \in \mathbb{C}^n$, $\mathbf{l} \neq \mathbf{0}$, is a pair of right and left nonzero starting vectors. The **nonsymmetric Lanczos process** [Lan50] is an extension of the symmetric Lanczos process that simultaneously constructs a nested basis for the Krylov subspace $K_j(C, \mathbf{r})$ induced by C and $\mathbf{r}$, and a nested basis for the Krylov subspace $K_j(C^T, \mathbf{l})$ induced by C^T and $\mathbf{l}$. In the context of the nonsymmetric Lanczos process, $K_j(C, \mathbf{r})$ is called the j th **right** Krylov subspace, and $K_j(C^T, \mathbf{l})$ is called the j th **left** Krylov subspace.

Algorithm (Nonsymmetric Lanczos process)

Compute $\rho_1 = \|\mathbf{r}\|_2$, $\eta_1 = \|\mathbf{l}\|_2$, and set $\mathbf{v}_1 = \mathbf{r}/\rho_1$, $\mathbf{w}_1 = \mathbf{l}/\eta_1$, $\mathbf{v}_0 = \mathbf{w}_0 = \mathbf{0}$, and $\delta_0 = 1$.

For $j = 1, 2, \dots$, do:

- 1) Compute $\delta_j = \mathbf{w}_j^T \mathbf{v}_j$.
If $\delta_j = 0$, stop.
- 2) Compute $\mathbf{v} = C\mathbf{v}_j$, and set $\beta_j = \eta_j \delta_j / \delta_{j-1}$ and $\mathbf{v} = \mathbf{v} - \mathbf{v}_{j-1} \beta_j$.
- 3) Compute $\alpha_j = \mathbf{w}_j^T \mathbf{v}$, and set $\mathbf{v} = \mathbf{v} - \mathbf{v}_j \alpha_j$.
- 4) Compute $\mathbf{w} = C^T \mathbf{w}_j$, and set $\gamma_j = \rho_j \delta_j / \delta_{j-1}$ and $\mathbf{w} = \mathbf{w} - \mathbf{w}_j \alpha_j - \mathbf{w}_{j-1} \gamma_j$.
- 5) Compute $\rho_{j+1} = \|\mathbf{v}\|_2$ and $\eta_{j+1} = \|\mathbf{w}\|_2$.
If $\rho_{j+1} = 0$ or $\eta_{j+1} = 0$, stop.
Otherwise, set $\mathbf{v}_{j+1} = \mathbf{v}/\rho_{j+1}$ and $\mathbf{w}_{j+1} = \mathbf{w}/\eta_{j+1}$.

end for

Facts:

The following facts can be found in [SB02, Sect. 8.7.3] or [Saa03, Sect. 7.1].

1. The occurrence of $\delta_j = 0$ in Step 1 of the nonsymmetric Lanczos process is called an **exact breakdown**. In finite-precision arithmetic, one also needs to check for $\delta_j \approx 0$, which is called a **near-breakdown**. It is possible to continue the nonsymmetric Lanczos process even if an exact breakdown or a near-breakdown has occurred, by using so-called “look-ahead” techniques. (See, e.g., [FGN93] and the references given there.) However, in practice, exact breakdowns and even near-breakdowns are fairly rare and, therefore, here we consider only the basic form of the nonsymmetric Lanczos process without look-ahead.
2. In exact arithmetic and if no exact breakdowns occur, the algorithm stops after a finite number of iterations. More precisely, it stops when $j = \min\{d(C, \mathbf{r}), d(C^T, \mathbf{l})\}$ is reached.
3. The **right Lanczos vectors** and the **left Lanczos vectors**

$$\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_j \quad \text{and} \quad \mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_j$$

generated during the first j iterations of the algorithm form a nested basis for the j th right Krylov subspace $K_j(C, \mathbf{r})$ and the j th left Krylov subspace $K_j(C^T, \mathbf{l})$, respectively.

4. The right and left Lanczos vectors satisfy the three-term recurrence relations

$$\mathbf{v}_{i+1} \rho_{i+1} = C\mathbf{v}_i - \mathbf{v}_i \alpha_i - \mathbf{v}_{i-1} \beta_i, \quad i = 1, 2, \dots, j,$$

and

$$\mathbf{w}_{i+1} \eta_{i+1} = C^T \mathbf{w}_i - \mathbf{w}_i \alpha_i - \mathbf{w}_{i-1} \gamma_i, \quad i = 1, 2, \dots, j,$$

respectively.

5. These three-term recurrence relations can be written in compact matrix form as follows:

$$\begin{aligned} CV_j &= V_j T_j + \rho_{j+1} \mathbf{v}_{j+1} \mathbf{e}_j^T = V_{j+1} T_j^{(e)}, \\ C^T W_j &= W_j \tilde{T}_j + \eta_{j+1} \mathbf{w}_{j+1} \mathbf{e}_j^T. \end{aligned}$$

Here, we set

$$\begin{aligned} V_j &= [\mathbf{v}_1 \quad \mathbf{v}_2 \quad \cdots \quad \mathbf{v}_j], \quad W_j = [\mathbf{w}_1 \quad \mathbf{w}_2 \quad \cdots \quad \mathbf{w}_j], \\ T_j &= \begin{bmatrix} \alpha_1 & \beta_2 & 0 & \cdots & 0 \\ \rho_2 & \alpha_2 & \beta_3 & \ddots & \vdots \\ 0 & \rho_3 & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta_j \\ 0 & \cdots & 0 & \rho_j & \alpha_j \end{bmatrix}, \quad \tilde{T}_j = \begin{bmatrix} \alpha_1 & \gamma_2 & 0 & \cdots & 0 \\ \eta_2 & \alpha_2 & \gamma_3 & \ddots & \vdots \\ 0 & \eta_3 & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \gamma_j \\ 0 & \cdots & 0 & \eta_j & \alpha_j \end{bmatrix}, \end{aligned}$$

$$\mathbf{e}_j^T = [0 \quad 0 \quad \cdots \quad 0 \quad 1] \in \mathbb{R}^{1 \times j}, \quad \text{and} \quad T_j^{(e)} = \begin{bmatrix} T_j \\ \rho_{j+1} \mathbf{e}_j^T \end{bmatrix}.$$

Note that $T_j, \tilde{T}_j \in \mathbb{C}^{j \times j}$, and $T_j^{(e)} \in \mathbb{C}^{(j+1) \times j}$ are tridiagonal matrices.

6. The matrix $T_j^{(e)}$ has full rank, i.e., $\text{rank } T_j^{(e)} = j$.
 7. In exact arithmetic, the right and left Lanczos vectors are **biorthogonal** to each other, i.e.,

$$\mathbf{w}_i^T \mathbf{v}_k = \begin{cases} 0 & \text{if } i \neq k, \\ \delta_i & \text{if } i = k, \end{cases} \quad \text{for all } i, k = 1, 2, \dots, j.$$

Since the right and left Lanczos vectors are the columns of V_j and W_j , respectively, the biorthogonality can be stated compactly as follows:

$$W_j^T V_j = D_j, \quad W_j^T \mathbf{v}_{j+1} = \mathbf{0}, \quad \text{and} \quad V_j^T \mathbf{w}_{j+1} = \mathbf{0}.$$

Here, D_j is the diagonal matrix

$$D_j = \text{diag}(\delta_1, \delta_2, \dots, \delta_j).$$

Note that D_j is nonsingular, as long as no exact breakdowns occur.

8. These biorthogonality relations, together with the above compact form of the three-term recurrence relations, imply that

$$T_j = D_j^{-1} V_j^* C V_j = (W_j^T V_j)^{-1} W_j^T C V_j.$$

Thus, the j th **Lanczos matrix** T_j is the oblique Petrov–Galerkin projection of C onto the j th right Krylov subspace $K_j(C, \mathbf{r})$, and orthogonally to the j th left Krylov subspace $K_j(C^T, \mathbf{l})$.

9. The matrices T_j and $\tilde{T}_j^T$ are diagonally similar:

$$\tilde{T}_j^T = D_j T_j D_j^{-1}.$$

10. The computational cost of each j th iteration of the nonsymmetric Lanczos process is fixed, and it is dominated by the matrix-vector product $\mathbf{v} = C \mathbf{v}_j$ with C and by the matrix-vector product $\mathbf{w} = C^T \mathbf{w}_j$ with C^T . In particular, the computational cost for generating the oblique Petrov–Galerkin projection T_j of C is dominated by the j matrix-vector products with C and the j matrix-vector products with C^T .

11. If C is a sparse matrix or a preconditioned matrix with a sparse preconditioner, then the matrix-vector products with C and C^T are fast. In this case, the nonsymmetric Lanczos process is a very efficient procedure for computing oblique Petrov–Galerkin projections T_j of C onto right Krylov subspaces $K_j(C, \mathbf{r})$, and orthogonally to left Krylov subspaces $K_j(C^T, \mathbf{l})$.
12. The three-term recurrence relations, which are used to generate the right and left Lanczos vectors, explicitly enforce biorthogonality only between three consecutive right vectors, $\mathbf{v}_{j-1}, \mathbf{v}_j, \mathbf{v}_{j+1}$, and three consecutive left vectors, $\mathbf{w}_{j-1}, \mathbf{w}_j, \mathbf{w}_{j+1}$. As a consequence, in finite-precision arithmetic, round-off error will usually cause loss of biorthogonality between all right vectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_{j+1}$ and all left vectors $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{j+1}$.
13. For applications of the Lanczos process in large-scale matrix computations, this loss of orthogonality is often benign, and only delays convergence. More precisely, in such applications, the Lanczos matrix $T_j \in \mathbb{C}^{j \times j}$ for some $j \ll n$ is used to obtain an approximate solution of a matrix problem involving the large matrix $C \in \mathbb{C}^{n \times n}$. Due to round-off error and the resulting loss of biorthogonality, the number j of iterations that is needed to obtain a satisfactory approximate solution is larger than the number of iterations that would be needed in exact arithmetic. (See, e.g., [CW86].)
14. If $C = C^*$ is a Hermitian matrix and $\mathbf{l} = \bar{\mathbf{r}}$, i.e., the left starting vector $\mathbf{l}$ is the complex conjugate of the right starting vector $\mathbf{r}$, then the right and left Lanczos vectors satisfy

$$\mathbf{w}_i = \bar{\mathbf{v}}_i \quad \text{for all } i = 1, 2, \dots, j+1,$$

and the nonsymmetric Lanczos process reduces to the symmetric Lanczos process.

49.6 The Arnoldi Process

The **Arnoldi process** [Arn51] is another extension of the symmetric Lanczos process for Hermitian matrices to general square matrices. Unlike the nonsymmetric Lanczos process, which produces bases for both right and left Krylov subspaces, the Arnoldi process generates basis vectors only for the right Krylov subspaces. However, these basis vectors are constructed to be orthonormal, resulting in a numerical procedure that is much more robust than the nonsymmetric Lanczos process.

In this section, we assume that $C \in \mathbb{C}^{n \times n}$ is a general square matrix, and that $\mathbf{r} \in \mathbb{C}^n$, $\mathbf{r} \neq \mathbf{0}$, is a nonzero starting vector.

Algorithm (Arnoldi process)

Compute $\rho_1 = \|\mathbf{r}\|_2$, and set $\mathbf{v}_1 = \mathbf{r}/\rho_1$.

For $j = 1, 2, \dots$, do:

- 1) Compute $\mathbf{v} = C\mathbf{v}_j$.
- 2) For $i = 1, 2, \dots, j$, do:
 - Compute $h_{ij} = \mathbf{v}^* \mathbf{v}_i$, and set $\mathbf{v} = \mathbf{v} - \mathbf{v}_i h_{ij}$.
 end for
- 3) Compute $h_{j+1,j} = \|\mathbf{v}\|_2$.
 If $h_{j+1,j} = 0$, stop.
 Otherwise, set $\mathbf{v}_{j+1} = \mathbf{v}/h_{j+1,j}$.

end for

Facts:

The following facts can be found in [Saa03, Sect. 6.3].

1. In exact arithmetic, the algorithm stops after a finite number of iterations. More precisely, it stops when $j = d(C, \mathbf{r})$ is reached.

2. The Arnoldi vectors

$$\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_j$$

generated during the first j iterations of the algorithm form a nested basis for the j th Krylov subspace $K_j(C, \mathbf{r})$.

3. The Arnoldi vectors satisfy the $(i + 1)$ -term recurrence relations

$$\mathbf{v}_{i+1}h_{i+1,i} = C\mathbf{v}_i - \mathbf{v}_ih_{ii} - \mathbf{v}_{i-1}h_{i-1,i} - \dots - \mathbf{v}_2h_{2i} - \mathbf{v}_1h_{1i}, \quad i = 1, 2, \dots, j.$$

These $(i + 1)$ -term recurrence relations can be written in compact matrix form as follows:

$$CV_j = V_jH_j + h_{j+1,j}\mathbf{v}_{j+1}\mathbf{e}_j^T = V_{j+1}H_j^{(e)}.$$

Here, we set

$$V_j = [\mathbf{v}_1 \quad \mathbf{v}_2 \quad \dots \quad \mathbf{v}_j], \quad \mathbf{e}_j^T = [0 \quad 0 \quad \dots \quad 0 \quad 1] \in \mathbb{R}^{1 \times j},$$

$$H_j = \begin{bmatrix} h_{11} & h_{12} & h_{13} & \dots & h_{1j} \\ h_{21} & h_{22} & h_{23} & \ddots & \vdots \\ 0 & h_{32} & \ddots & \ddots & h_{j-2,j} \\ \vdots & \ddots & \ddots & \ddots & h_{j-1,j} \\ 0 & \dots & 0 & h_{j,j-1} & h_{jj} \end{bmatrix},$$

$$H_j^{(e)} = \begin{bmatrix} H_j \\ h_{j+1,j}\mathbf{e}_j^T \end{bmatrix}, \quad \text{and} \quad V_{j+1} = [V_j \quad \mathbf{v}_{j+1}].$$

Note that $H_j \in \mathbb{C}^{j \times j}$ and $H_j^{(e)} \in \mathbb{C}^{(j+1) \times j}$ are upper Hessenberg matrices.

4. The matrix $H_j^{(e)}$ has full rank, i.e., $\text{rank } H_j^{(e)} = j$.
 5. Since the Arnoldi vectors are the columns of V_j , this orthonormality can be stated compactly as follows:

$$V_j^* V_j = I_j \quad \text{and} \quad V_j^* \mathbf{v}_{j+1} = \mathbf{0}.$$

6. These orthogonality relations, together with the above compact form of the recurrence relations, imply that

$$H_j = V_j^* C V_j.$$

Thus, the j th **Arnoldi matrix** H_j is the orthogonal Petrov–Galerkin projection of C onto the j th Krylov subspace $K_j(C, \mathbf{r})$.

7. As in the case of the symmetric Lanczos process, each j th iteration of the Arnoldi process requires only a single matrix-vector product $\mathbf{v} = C\mathbf{v}_j$. If C is a sparse matrix or a preconditioned matrix with a sparse preconditioner, then the matrix-vector products with C are fast.
 8. However, unlike the Lanczos process, the additional computations in each j th iteration do increase with j . In particular, each j th iteration requires the computation of j inner products of vectors of length n , and the computation of j SAXPY-type updates of the form $\mathbf{v} = \mathbf{v} - \mathbf{v}_j h_{ij}$ with vectors of length n .
 9. For most large-scale matrix computations, the increasing work per iteration limits the number of iterations that the Arnoldi process can be run. Therefore, in practice, the Arnoldi process is usually combined with restarting; i.e., after a number of iterations (with the matrix C and starting vector $\mathbf{r}$), the algorithm is started again with the same matrix C , but a different starting vector, say $\mathbf{r}_1$.
 10. On the other hand, the $(i + 1)$ -term recurrence relations used to generate the Arnoldi vectors explicitly enforce orthogonality among the first $i + 1$ vectors, $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_{i+1}$. As a result, the Arnoldi process is much less susceptible to round-off error in finite-precision arithmetic than the Lanczos process.

49.7 Eigenvalue Computations

In this section, we consider the problem of computing a few eigenvalues, and possibly eigenvectors, of a large matrix $C \in \mathbb{C}^{n \times n}$. We assume that matrix-vector products with C are fast. In this case, orthogonal and, in the non-Hermitian case, oblique Petrov–Galerkin projections of C onto Krylov subspaces $K_j(C, \mathbf{r})$ can be computed efficiently, as long as $j \ll n$.

Facts:

The following facts can be found in [CW85], [CW86], and [BDD00].

1. Assume that $C = C^* \in \mathbb{C}^{n \times n}$ is a Hermitian matrix. We choose any nonzero starting vector $\mathbf{r} \in \mathbb{C}^n$, $\mathbf{r} \neq \mathbf{0}$, e.g., a vector with random entries, and run the symmetric Lanczos process. After j iterations of the algorithm, we have computed the j th Lanczos matrix T_j , which — in exact arithmetic — is the orthogonal Petrov–Galerkin projection of C onto the j th Krylov subspace $K_j(C, \mathbf{r})$. Neglecting the last term in the compact form of the three-term recurrence relations used in the first j iterations of the symmetric Lanczos process, we obtain the approximation

$$CV_j \approx V_j T_j.$$

This approximation suggests to use the j eigenvalues $\lambda_i^{(j)}$, $i = 1, 2, \dots, j$, of the j th Lanczos matrix $T_j \in \mathbb{C}^{j \times j}$ as approximate eigenvalues of the original matrix C . Furthermore, if one is also interested in approximate eigenvectors, then the above approximation suggests to use

$$\mathbf{x}_i^{(j)} = V_j \mathbf{z}_i^{(j)} \in \mathbb{C}^n, \quad \text{where} \quad T_j \mathbf{z}_i^{(j)} = \mathbf{z}_i^{(j)} \lambda_i^{(j)}, \quad \mathbf{z}_i^{(j)} \neq \mathbf{0},$$

as an approximate eigenvector of C corresponding to the approximate eigenvalue $\lambda_i^{(j)}$ of C .

2. Assume that $C \in \mathbb{C}^{n \times n}$ is a general square matrix. Here one can use either the nonsymmetric Lanczos process or the Arnoldi process to obtain approximate eigenvalues.
3. In the case of the nonsymmetric Lanczos process, one chooses any nonzero starting vectors $\mathbf{r} \in \mathbb{C}^n$, $\mathbf{r} \neq \mathbf{0}$, and $\mathbf{l} \in \mathbb{C}^n$, $\mathbf{l} \neq \mathbf{0}$, $\mathbf{r} \in \mathbb{C}^n$, $\mathbf{r} \neq \mathbf{0}$. In analogy to the symmetric case, the eigenvalues of Lanczos matrix $T_j \in \mathbb{C}^{j \times j}$ computed by j iterations of the nonsymmetric Lanczos process are used as approximate eigenvalues of the original matrix C . Corresponding approximate right eigenvectors are given by the same formula as above. Furthermore, one can also obtain approximate left eigenvectors from the left eigenvectors of T_j and the first j left Lanczos vectors. A discussion of many practical aspects of using the nonsymmetric Lanczos process for eigenvalue computations can be found in [CW86].
4. In the case of the Arnoldi process, one only needs to choose a single nonzero starting vector $\mathbf{r} \in \mathbb{C}^n$, $\mathbf{r} \neq \mathbf{0}$. Here, one has the approximation

$$CV_j \approx V_j H_j,$$

where V_j is the matrix containing the first j Arnoldi vectors as columns and H_j is the j th Arnoldi matrix. The eigenvalues $\lambda_i^{(j)}$, $i = 1, 2, \dots, j$, of $H_j \in \mathbb{C}^{j \times j}$ are used as approximate eigenvalues of C . Furthermore, for each i ,

$$\mathbf{x}_i^{(j)} = V_j \mathbf{z}_i^{(j)} \in \mathbb{C}^n, \quad \text{where} \quad H_j \mathbf{z}_i^{(j)} = \mathbf{z}_i^{(j)} \lambda_i^{(j)}, \quad \mathbf{z}_i^{(j)} \neq \mathbf{0},$$

is an approximate eigenvector of C corresponding to the approximate eigenvalue $\lambda_i^{(j)}$ of C .

49.8 Linear Systems of Equations

In this section, we consider the problem of solving large systems of linear equations,

$$C\mathbf{x} = \mathbf{b},$$

where $C \in \mathbb{C}^{n \times n}$ is a nonsingular matrix and $\mathbf{b} \in \mathbb{C}^n$. We assume that any possible preconditioning was already applied and so, in general, C is a preconditioned version of the original coefficient matrix.

In particular, the matrix C may actually be dense. However, we assume that matrix-vector products with C and possibly C^T are fast. This is the case when C is a preconditioned version of a sparse matrix A and a preconditioner A_0 that allows a sparse LU or Cholesky factorization.

Facts:

The following facts can be found in [FGN92] or [Saa03].

1. Let $\mathbf{x}_0 \in \mathbb{C}^n$ be an arbitrary initial guess for the solution of the linear system, and denote by

$$\mathbf{r}_0 = \mathbf{b} - C\mathbf{x}_0$$

the corresponding residual vector. A Krylov subspace-based iterative method for the solution of the above linear system constructs a sequence of approximate solutions of the form

$$\mathbf{x}_j \in \mathbf{x}_0 + K_j(C, \mathbf{r}_0), \quad j = 1, 2, \dots,$$

i.e., the j th iterate is an additive correction of the initial guess, where the correction is chosen from the j th Krylov subspace $K_j(C, \mathbf{r}_0)$ induced by the coefficient matrix C and the initial residual $\mathbf{r}_0$. Now let $V_j \in \mathbb{C}^{n \times j}$ be a matrix the columns of which form a nested basis for $K_j(C, \mathbf{r}_0)$. Then, any possible j th iterate can be parametrized in the form

$$\mathbf{x}_j = \mathbf{x}_0 + V_j \mathbf{z}_j, \quad \text{where } \mathbf{z}_j \in \mathbb{C}^j.$$

Moreover, the corresponding residual vector is given by

$$\mathbf{r}_j = \mathbf{b} - C\mathbf{x}_j = \mathbf{r}_0 - C V_j \mathbf{z}_j.$$

Different Krylov subspace-based iterative methods are then obtained by specifying the choice of the basis matrix V_j and the choice of the parameter vector $\mathbf{z}_j$.

2. The **biconjugate gradient algorithm** (BCG) [Lan52] employs the nonsymmetric Lanczos process to generate nested bases for the right Krylov subspaces $K_j(C, \mathbf{r}_0)$ and the left Krylov subspaces $K_j(C^T, \mathbf{l})$. Here, $\mathbf{l} \in \mathbb{C}^n$, $\mathbf{l} \neq \mathbf{0}$, is an arbitrary nonzero starting vector. The biorthogonality of the right and left Lanczos vectors is exploited to construct the j th iterate $\mathbf{x}_j$ such that the corresponding residual vector $\mathbf{r}_j$ is orthogonal to the left Lanczos vectors, i.e., $W_j^T \mathbf{r}_j = \mathbf{0}$. Using the recurrence relations of the Lanczos process and the above relation for $\mathbf{r}_j$, one can show that the defining condition $W_j^T \mathbf{r}_j = \mathbf{0}$ is equivalent to $\mathbf{z}_j$ being the solution of the linear system

$$T_j \mathbf{z}_j = \mathbf{e}_1^{(j)} \rho_1,$$

where $\mathbf{e}_1^{(j)}$ denotes the first unit vector of length j . Moreover, the corresponding iterates $\mathbf{x}_j$ can be obtained via a simple update from the previous iterate $\mathbf{x}_{j-1}$, resulting in an elegant overall computational procedure. Unfortunately, in general, it cannot be guaranteed that all Lanczos matrices T_j are nonsingular. As a result, BCG iterates $\mathbf{x}_j$ may not exist for every j . More precisely, BCG breaks down if T_j is singular, and it exhibits erratic convergence behavior when T_j is nearly singular.

3. The possible breakdowns and the erratic convergence behavior can be avoided by replacing the $j \times j$ linear system $T_j \mathbf{z}_j = \mathbf{e}_1^{(j)} \rho_1$ by the $(j+1) \times j$ least-squares problem

$$\min_{\mathbf{z} \in \mathbb{C}^j} \left\| \mathbf{e}_1^{(j+1)} \rho_1 - T_j^{(e)} \mathbf{z} \right\|_2.$$

Since $T_j^{(e)} \in \mathbb{C}^{(j+1) \times j}$ always has full rank j , the above least-squares problem has a unique solution $\mathbf{z}_j$. The resulting iterative procedure is the **quasi-minimal residual method** (QMR) [FN91].

4. The **generalized minimal residual algorithm** (GMRES) [SS86] uses the Arnoldi process to generate orthonormal basis vectors for the Krylov subspaces $K_j(C, \mathbf{r}_0)$. The orthonormality of the columns of the Arnoldi basis matrix V_j allows one to choose $\mathbf{z}_j$ such that the residual vector $\mathbf{r}_j$ has the smallest possible norm, i.e.,

$$\|\mathbf{r}_j\|_2 = \|\mathbf{r}_0 - C V_j \mathbf{z}_j\|_2 = \min_{\mathbf{z} \in \mathbb{C}^j} \|\mathbf{r}_0 - C V_j \mathbf{z}\|_2.$$

Using the compact form of the recurrence relations used to generate the Arnoldi vectors, one readily verifies that the above minimal residual property is equivalent to $\mathbf{z}_j$ being the solution of the least-squares problem

$$\min_{\mathbf{z} \in \mathbb{C}^j} \left\| \mathbf{e}_1^{(j+1)} \rho_1 - H_j^{(e)} \mathbf{z} \right\|_2,$$

where $H_j^{(e)} \in \mathbb{C}^{(j+1) \times j}$ is an upper Hessenberg matrix.

5. The idea of quasi-minimization of the residual vector can also be applied to Lanczos-type iterations that, in each j th step, perform two matrix–vector products with C , instead of one product with C and one product with C^T . The resulting algorithm is called the **transpose-free quasi-minimal residual method** (TFQMR) [Fre93]. We stress that QMR and TFQMR produce different sequences of iterates and, thus, QMR and TFQMR are not mathematically equivalent algorithms.

49.9 Dimension Reduction of Linear Dynamical Systems

In this section, we discuss the application of the nonsymmetric Lanczos process to a large-scale matrix problem that arises in dimension reduction of time-invariant linear dynamical systems. A more detailed description can be found in [Fre03].

Definitions:

Let $A, E \in \mathbb{C}^{n \times n}$. The matrix pencil $A - sE$, $s \in \mathbb{C}$, is said to be **regular** if the matrix $A - sE$ is singular only for finitely many values $s \in \mathbb{C}$.

A **single-input, single-output, time-invariant linear dynamical system** is a system of differential-algebraic equations (DAEs) of the form

$$\begin{aligned} E \frac{d}{dt} \mathbf{x} &= A\mathbf{x} + \mathbf{b}u(t), \\ y(t) &= \mathbf{l}^T \mathbf{x}(t), \end{aligned}$$

together with suitable initial conditions. Here, $A, E \in \mathbb{C}^{n \times n}$ are given matrices such that $A - sE$ is a regular matrix pencil, $\mathbf{b} \in \mathbb{C}^n$, $\mathbf{b} \neq \mathbf{0}$, and $\mathbf{l} \in \mathbb{C}^n$, $\mathbf{l} \neq \mathbf{0}$, are given nonzero vectors, $\mathbf{x}(t) \in \mathbb{C}^n$ is the vector of **state variables**, $u(t) \in \mathbb{C}$ is the given input function, $y(t) \in \mathbb{C}$ is the output function, and n is the **state-space dimension**.

The rational function

$$H : \mathbb{C} \mapsto \mathbb{C} \cup \infty, \quad H(s) := \mathbf{l}^T (sE - A)^{-1} \mathbf{b},$$

is called the **transfer function** of the above time-invariant linear dynamical system.

A **reduced-order model** of state-space dimension j ($< n$) of the above system is a single-input, single-output, time-invariant linear dynamical system of the form

$$\begin{aligned} E_j \frac{d}{dt} \mathbf{z} &= A_j \mathbf{z} + \mathbf{b}_j u(t), \\ y(t) &= \mathbf{l}_j^T \mathbf{z}(t), \end{aligned}$$

where $A_j, E_j \in \mathbb{C}^{j \times j}$ and $\mathbf{b}_j, \mathbf{l}_j \in \mathbb{C}^j$, together with suitable initial conditions.

Let $s_0 \in \mathbb{C}$ be such that the matrix $A - s_0 E$ is nonsingular. A reduced-order model of state-space dimension j of the above system is said to be a **Padé model** about the expansion point s_0 if the matrices A_j, E_j and the vectors $\mathbf{b}_j, \mathbf{l}_j$ are chosen such that the Taylor expansions about s_0 of the transfer function H of the original system and of the reduced-order transfer function

$$H_j : \mathbb{C} \mapsto \mathbb{C} \cup \infty, \quad H_j(s) := \mathbf{l}_j^T (sE_j - A_j)^{-1} \mathbf{b}_j,$$

agree in as many leading Taylor coefficients as possible, i.e.,

$$H_j(s) = H(s) + \mathcal{O}((s - s_0)^{q(j)}),$$

where $q(j)$ is as large as possible.

Facts:

The following facts can be found in [FF94], [FF95], or [Fre03].

1. In the “generic” case, $q(j) = 2j$.
2. In the general case, $q(j) \geq 2j$; the case $q(j) > 2j$ occurs only in certain degenerate situations.
3. The transfer function H can be rewritten in terms of a single square matrix $C \in \mathbb{C}^{n \times n}$ as follows:

$$H(s) = \mathbf{1}^T (I_n + (s - s_0)C)^{-1} \mathbf{r}, \quad \text{where } C := (s_0 E - A)^{-1} E, \quad \mathbf{r} := (s_0 E - A)^{-1} \mathbf{b}.$$

Note that the matrix C can be viewed as a preconditioned version of the matrix E using the “shift-and-invert” preconditioner $s_0 E - A$.

4. In many cases, the state-space dimension n of the original time-invariant linear dynamical system is very large, but the large square matrices $A, E \in \mathbb{C}^{n \times n}$ are sparse. Furthermore, these matrices are usually such that sparse LU factorizations of the shift-and-invert preconditioner $s_0 E - A$ can be computed with limited amounts of fill-in. In this case, matrix-vector products with the preconditioned matrix C and its transpose C^T are fast.
5. The above definition of Padé models suggests the computation of these reduced-order models by first explicitly generating the leading $q(j)$ Taylor coefficients of H about the expansion point s_0 , and then constructing the Padé model from these. However, this process is extremely ill-conditioned and numerically unstable. (See the discussion in [FF94] or [FF95].)
6. A much more stable way to compute Padé models without explicitly generating the Taylor coefficients is based on the nonsymmetric Lanczos process. The procedure is simply as follows: One uses the vectors $\mathbf{r}$ and $\mathbf{l}$ from the above representation of the transfer function H as right and left starting vectors, and applies the nonsymmetric Lanczos process to the preconditioned matrix C . After j iterations, the algorithm has produced the $j \times j$ tridiagonal Lanczos matrix T_j . The reduced-order model defined by

$$A_j := s_0 T_j - I_j, \quad E_j := T_j, \quad \mathbf{b}_j := (\mathbf{l}^T \mathbf{r}) \mathbf{e}_1^{(j)}, \quad \mathbf{l}_j := \mathbf{e}_1^{(j)}$$

is a Padé model of state-space dimension j about the expansion point s_0 . Here, $\mathbf{e}_1^{(j)}$ denotes the first unit vector of length j .

7. In the large-scale case, Padé models of state-space dimension $j \ll n$ often provide very accurate approximations of the original system of state-space dimension n . In particular, this is the case for applications in VLSI circuit simulation. (See [FF95], [Fre03], and the references given there.)
8. Multiple-input multiple-output time-invariant linear dynamical systems are extensions of the above single-input single-output case with the vectors $\mathbf{b}$ and $\mathbf{l}$ replaced by matrices $B \in \mathbb{C}^{n \times m}$ and $L \in \mathbb{C}^{n \times p}$, respectively, where m is the number of inputs and p is the number of outputs. The approach outlined in this section can be extended to the general multiple-input multiple-output case. A suitable extension of the nonsymmetric Lanczos process that can handle multiple right and left starting vectors is needed in this case. For a discussion of such a Lanczos-type algorithm and its application in dimension reduction of general multiple-input multiple-output time-invariant linear dynamical systems, we refer the reader to [Fre03] and the references given there.

References

- [Arn51] W.E. Arnoldi. The principle of minimized iterations in the solution of the matrix eigenvalue problem. *Quart. Appl. Math.*, 9:17–29, 1951.
- [BDD00] Z. Bai, J. Demmel, J. Dongarra, A. Ruhe, and H. van der Vorst, Eds., *Templates for the Solution of Algebraic Eigenvalue Problems: A Practical Guide*. SIAM Publications, Philadelphia, PA, 2000.
- [CW85] J.K. Cullum and R.A. Willoughby. *Lanczos Algorithms for Large Symmetric Eigenvalue Computations, Vol. 1, Theory*. Birkhäuser, Basel, Switzerland, 1985.
- [CW86] J.K. Cullum and R.A. Willoughby. A practical procedure for computing eigenvalues of large sparse nonsymmetric matrices. In J.K. Cullum and R.A. Willoughby, Eds., *Large Scale Eigenvalue Problems*, pp. 193–240. North-Holland, Amsterdam, The Netherlands, 1986.
- [DER89] I.S. Duff, A.M. Erisman, and J.K. Reid. *Direct Methods for Sparse Matrices*. Oxford University Press, Oxford, U.K., 1989.
- [FF94] P. Feldmann and R.W. Freund. Efficient linear circuit analysis by Padé approximation via the Lanczos process. In *Proceedings of EURO-DAC '94 with EURO-VHDL '94*, pp. 170–175, Los Alamitos, CA, 1994. IEEE Computer Society Press.
- [FF95] P. Feldmann and R.W. Freund. Efficient linear circuit analysis by Padé approximation via the Lanczos process. *IEEE Trans. Comp.-Aid. Des.*, 14:639–649, 1995.
- [Fre93] R.W. Freund. A transpose-free quasi-minimal residual algorithm for non-Hermitian linear systems. *SIAM J. Sci. Comp.*, 14:470–482, 1993.
- [Fre03] R.W. Freund. Model reduction methods based on Krylov subspaces. *Acta Numerica*, 12:267–319, 2003.
- [FGN92] R.W. Freund, G.H. Golub, and N.M. Nachtigal. Iterative solution of linear systems. *Acta Numerica*, 1:57–100, 1992.
- [FGN93] R.W. Freund, M.H. Gutknecht, and N.M. Nachtigal. An implementation of the look-ahead Lanczos algorithm for non-Hermitian matrices. *SIAM J. Sci. Comp.*, 14:137–158, 1993.
- [FN91] R.W. Freund and N.M. Nachtigal. QMR: a quasi-minimal residual method for non-Hermitian linear systems. *Num. Math.*, 60:315–339, 1991.
- [GMS05] P.E. Gill, W. Murray, and M.A. Saunders. SNOPT: An SQP algorithm for large-scale constrained optimization. *SIAM Rev.*, 47:99–131, 2005.
- [Hou75] A.S. Householder. *The Theory of Matrices in Numerical Analysis*. Dover Publications, New York, 1975.
- [KR91] N.K. Karmarkar and K.G. Ramakrishnan. Computational results of an interior point algorithm for large scale linear programming. *Math. Prog.*, 52:555–586, 1991.
- [Lan50] C. Lanczos. An iteration method for the solution of the eigenvalue problem of linear differential and integral operators. *J. Res. Nat. Bur. Stand.*, 45:255–282, 1950.
- [Lan52] C. Lanczos. Solution of systems of linear equations by minimized iterations. *J. Res. Nat. Bur. Stand.*, 49:33–53, 1952.
- [LM05] A.N. Langville and C.D. Meyer. A survey of eigenvector methods for web information retrieval. *SIAM Rev.*, 47(1):135–161, 2005.
- [NW99] J. Nocedal and S.J. Wright. *Numerical Optimization*. Springer-Verlag, New York, 1999.
- [Saa03] Y. Saad. *Iterative Methods for Sparse Linear Systems*. SIAM Publications, Philadelphia, PA, 2nd ed., 2003.
- [SS86] Y. Saad and M.H. Schultz. GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM J. Sci. Statist. Comp.*, 7(3):856–869, 1986.
- [SB02] J. Stoer and R. Bulirsch. *Introduction to Numerical Analysis*. Springer-Verlag, New York, 3rd ed., 2002.
- [vdV03] H.A. van der Vorst. *Iterative Krylov Methods for Large Linear Systems*. Cambridge University Press, Cambridge, 2003.
- [Wri97] S.J. Wright. *Primal-dual interior-point methods*. SIAM Publications, Philadelphia, PA, 1997.

61	Coding Theory	<i>Joachim Rosenthal</i> <i>and Paul Weiner</i>	61-1
62	Quantum Computation	<i>Zijian Diao</i>	62-1
63	Information Retrieval and Web Search	<i>Amy N. Langville</i> <i>and Carl D. Meyer</i>	63-1
64	Signal Processing	<i>Michael Stewart</i>	64-1

Applications to Geometry

- 65 **Geometry** *Mark Hunacek* 65-1
- 66 **Some Applications of Matrices and Graphs in Euclidean Geometry** *Miroslav Fiedler* 66-1

Applications to Algebra

- 67 **Matrix Groups** *Peter J. Cameron* 67-1
- 68 **Group Representations** *Randall Holmes and T. Y. Tam* 68-1
- 69 **Nonassociative Algebras** *Murray R. Bremner, Lucia I. Murakami, and Ivan P. Shestakov* 69-1
- 70 **Lie Algebras** *Robert Wilson* 70-1

Applications to Optimization

50	Linear Programming	<i>Leonid N. Vaserstein</i>	50-1
	What Is Linear Programming? • Setting Up (Formulating) Linear Programs		
	• Standard and Canonical Forms for Linear Programs • Standard Row		
	Tableaux • Pivoting • Simplex Method • Geometric Interpretation of		
	Phase 2 • Duality • Sensitivity Analysis and Parametric Programming • Matrix		
	Games • Linear Approximation • Interior Point Methods		
51	Semidefinite Programming	<i>Henry Wolkowicz</i>	51-1
	Introduction • Specific Notation and Preliminary Results • Geometry		
	• Duality and Optimality Conditions • Strong Duality without a Constraint		
	Qualification • A Primal-Dual Interior-Point Algorithm • Applications of SDP		

50

Linear Programming

50.1	What Is Linear Programming?	50-1
50.2	Setting Up (Formulating) Linear Programs	50-3
50.3	Standard and Canonical Forms for Linear Programs	50-7
50.4	Standard Row Tableaux	50-8
50.5	Pivoting	50-10
50.6	Simplex Method	50-11
50.7	Geometric Interpretation of Phase 2	50-13
50.8	Duality	50-13
50.9	Sensitivity Analysis and Parametric Programming ..	50-17
50.10	Matrix Games	50-18
50.11	Linear Approximation	50-20
50.12	Interior Point Methods	50-23
	References	50-24

Leonid N. Vaserstein
Penn State

We freely use the textbook [Vas03]. Additional references, including references to Web sites with software, can be found in [Vas03], [Ros00, Sec. 15.1], and INFORMS Resources (<http://www.informs.org/Resources/>).

50.1 What Is Linear Programming?

Definitions:

Optimization is maximization or minimization of a real-valued function, called the **objective function**, on a set, called the **feasible region** or the set of **feasible solutions**.

The values of function on the set are called **feasible values**.

An **optimal solution** (optimizer) is a feasible solution where the objective function reaches an **optimal value** (optimum), i.e., maximal or minimal value, respectively.

An optimization problem is **infeasible** if there are no feasible solutions, i.e., the feasible region is empty.

It is **unbounded** if the feasible values are arbitrary large, in the case of a maximization problem, or arbitrary small, in the case of a minimization problem.

A **mathematical program** is an optimization problem where the feasible region is a subset of $\mathbb{R}^n$, a finite dimensional real space; i.e., the objective function is a function of one or several real variables.

A **linear form** in variables $x_1, \dots, x_n$ is $c_1x_1 + \dots + c_nx_n$, where c_i are given numbers.

An **affine function** is a linear form plus a given number.

The term *linear function*, which is not used here, means a linear form in some textbooks and an affine function in others.

A **linear constraint** is one of the following three constraints: $f \leq g$, $f = g$, $f \geq g$, where f and g are affine functions. In standard form, f is a linear form and g is a given number.

A **linear program** is a mathematical program where the objective function is an affine function and the feasible region is given by a finite system of linear constraints, all of them to be satisfied.

Facts:

For background reading on the material in this subsection see [Vas03].

1. Linear constraints with equality signs are known as linear equations. The main tool of simplex method, pivot steps, allows us to solve any system of linear equations.
2. In some textbooks, the objective function in a linear program is required to be a linear form. Dropping a constant in the objective function does not change optimal solutions, but the optimal value changes in the obvious way.
3. Solving an optimization problem usually means finding the optimal value and an optimal solution or showing that they do not exist. By comparison, solving a system of linear equations usually means finding all solutions. In both cases, in real life we only find approximate solutions.
4. Linear programs with one and two variables can be solved graphically. In the case of one variable x , the feasible region has one of the following forms: Empty, a point $x = a$, a finite interval $a \leq x \leq b$, with $a < b$, a ray $x \geq a$ or $x \leq a$, the whole line. The objective function $f = cx + d$ is represented by a straight line. Depending on the sign of c and whether we want maximize or minimize f , we move in the feasible region to the right or to the left as far as we can in search for an optimal solution.

In the case of two variables, the feasible region is a closed convex set with finitely many vertices (corners). Together with the feasible region, we can draw in plane levels of the objective function. Unless the objective function is constant, every level (where the function takes a certain value) is a straight line. This picture allows us to see whether the program is feasible and bounded. If it is, the picture allows us to find a vertex which is optimal.

5. Linear programming is about formulating, collecting data, and solving linear programs, and also about analyzing and implementing solutions in real life.
6. Linear programming is an important part of mathematical programming. In its turn, mathematical programming is a part of *operations research*. *Systems engineering* and *management science* are engineering and business versions of operations research.
7. Every linear program is either infeasible, or unbounded, or has an optimal solution.

Examples:

1. Here are 3 linear forms in x, y, z : $2x - 3y + 5z, x + z, y$.
2. Here are 3 affine functions of x, y, z : $2x - 3y + 5z - 1, x + z + 3, y$.
3. Here are 3 functions of x, y, z that are not affine: $xy, x^2 + z^3, \sin z$.
4. The constraint $|x| \leq 1$ is not linear, but it is equivalent to a system of two linear constraints, $x \leq 1, x \geq -1$.
5. (An infeasible linear program) Here is a linear program:
Maximize $x + y$ subject to $x \leq -1, x \geq 0$.
It has two variables and two linear constraints. The objective function is a linear form. The program is infeasible.
6. (An unbounded linear program) Here is a linear program:
Maximize $x + y$.
This program has two variables and no constraints. The objective function is a linear form. The program is unbounded.
7. Here is a linear program:
Minimize $x + y$ subject to $x \geq 1, y \geq 2$.
This program has two variables and two linear constraints. The objective function is a linear form. The optimal value (the maximum) is 3. An optimal solution is $x = 1, y = 2$. It is unique.

50.2 Setting Up (Formulating) Linear Programs

Examples:

1. Finding the maximum of n given numbers $c_1, \dots, c_n$ does not look like a linear program. However, it is equivalent to the following linear program with n variables x_i and $n + 1$ linear constraints:
 $c_1x_1 + \dots + c_nx_n \rightarrow \max$, all $x_i \geq 0$, $x_1 + \dots + x_n = 1$.

An equivalent linear program with one variable y and n linear constraints is $y \rightarrow \min$, $y \geq c_i$ for all i .

2. (Diet problem [Vas03, Ex. 2.1]). The general idea is to select a mix of different foods in such a way that basic nutritional requirements are satisfied at minimum cost. Our example is drastically simplified.

According to the recommendations of a nutritionist, a person's daily requirements for protein, vitamin A, and calcium are as follows: 50 grams of protein, 4000 IUs (international units) of vitamin A, 1000 milligrams of calcium. For illustrative purposes, let us consider a diet consisting only of apples (raw, with skin), bananas (raw), carrots (raw), dates (domestic, natural, pitted, chopped), and eggs (whole, raw, fresh) and let us, if we can, determine the amount of each food to be consumed in order to meet the recommended dietary allowances (RDA) at minimal cost.

Food	Unit	Protein (g)	Vit. A (IU)	Calcium (mg)
Apple	1 medium (138 g)	0.3	73	9.6
Banana	1 medium (118 g)	1.2	96	7
Carrot	1 medium (72 g)	0.7	20253	19
Dates	1 cup (178 g)	3.5	890	57
Egg	1 medium (44 g)	5.5	279	22

Since our goal is to meet the RDA with minimal cost, we also need to compile the costs of these foods:

Food	Cost (in cents)
1 apple	10
1 banana	15
1 carrot	5
1 cup dates	60
1 egg	8

Using these data, we can now set up a linear program. Let a, b, c, d, e be variables representing the quantities of the five foods we are going to use in the diet. The objective function to be minimized is the total cost function (in cents),

$$C = 10a + 15b + 5c + 60d + 8e,$$

where the coefficients represent cost per unit of the five items under consideration.

What are the constraints? Obviously,

$$a, b, c, d, e \geq 0. \quad (i)$$

These constraints are called *nonnegativity constraints*.

Then, to ensure that the minimum daily requirements of protein, vitamin A, and calcium are satisfied, it is necessary that

$$\begin{cases} 0.3a + 1.2b + 0.7c + 3.5d + 5.5e \geq 50 \\ 73a + 96b + 20253c + 890d + 279e \geq 4000 \\ 9.6a + 7b + 19c + 57d + 22e \geq 1000, \end{cases} \quad (ii)$$

where, for example, in the first constraint, the term $0.3a$ expresses the number of grams of protein in each apple multiplied by the quantity of apples needed in the diet, the second term $1.2b$ expresses the number of grams of protein in each banana multiplied by the quantity of bananas needed in the diet, and so forth.

Thus, we have a linear program with 5 variables and 8 linear constraints.

3. (Blending problem [Vas03, Ex. 2.2]). Many coins in different countries are made from cupronickel (75% copper, 25% nickel). Suppose that the four available alloys (scrap metals) A , B , C , D to be utilized to produce the coin contain the percentages of copper and nickel shown in the following table:

Alloy	A	B	C	D
% copper	90	80	70	60
% nickel	10	20	30	40
\$/lb	1.2	1.4	1.7	1.9

The cost in dollars per pound of each alloy is given as the last row in the same table.

Notice that none of the four alloys contains the desired percentages of copper and nickel. Our goal is to combine these alloys into a new blend containing the desired percentages of copper and nickel for cupronickel while minimizing the cost. This lends itself to a linear program.

Let a , b , c , d be the amounts of alloys A , B , C , D in pounds to make a pound of the new blend. Thus,

$$a, b, c, d \geq 0. \quad (i)$$

Since the new blend will be composed exclusively from the four alloys, we have

$$a + b + c + d = 1. \quad (ii)$$

The conditions on the composition of the new blend give

$$\begin{cases} .9a + .8b + .7c + .6d = .75 \\ .1a + .2b + .3c + .4d = .25. \end{cases} \quad (iii)$$

For example, the first equality states that 90% of the amount of alloy A , plus 80% of the amount of alloy B , plus 70% of the amount of alloy C , plus 60% of the amount of alloy D will give the desired 75% of copper in a pound of the new blend. Likewise, the second equality gives the desired amount of nickel in the new blend.

Taking the preceding constraints into account, we minimize the cost function

$$C = 1.2a + 1.4b + 1.7c + 1.9d.$$

In this problem, all the constraints, except (i), are *equalities*. In fact, there are three linear equations and four unknowns. However, the three equations are not independent. For example, the sum of the equations in (iii) gives (ii). Thus, (ii) is redundant.

In general, a constraint is said to be *redundant* if it follows from the other constraints of our system. Since it contributes no new information regarding the solutions of the linear program, it can be dropped from consideration without changing the feasible set.

4. (Manufacturing problem [Vas03, Ex. 2.3]). We are now going to state a program in which the objective function, a profit function, is to be maximized. A factory produces three products: P_1 , P_2 , and P_3 . The unit of measure for each product is the standard-sized boxes into which the product is placed. The profit per box of P_1 , P_2 , and P_3 is \$2, \$3, and \$7, respectively. Denote by x_1 , x_2 , x_3 the number of boxes of P_1 , P_2 , and P_3 , respectively. So the profit function we want to maximize is

$$P = 2x_1 + 3x_2 + 7x_3.$$

The five resources used are raw materials R1 and R2, labor, working area, and time on a machine. There are 1200 lbs of R1 available, 300 lbs of R2, 40 employee-hours of labor, 8000 m² of working area, and 8 machine-hours on the machine.

The amount of each resource needed for a box of each of the products is given in the following table (which also includes the aforementioned data):

Resource	Unit	P1	P2	P3	Available
R1	lb	40	20	60	1200
R2	lb	4	1	6	300
Labor	hour	.2	.7	2	40
Area	m ²	100	100	800	8000
Machine	hour	.1	.3	.6	8
Profit	\$	2	3	7	→ max

As we see from this table, to produce a box of P1 we need 40 pounds of R1, 4 pounds of R2, 0.2 hours of labor, 100 m² of working area, and 0.1 hours on the machine. Also, the amount of resources needed to produce a box of P2 and P3 can be deduced from the table. The constraints are

$$x_1, x_2, x_3 \geq 0, \quad (i)$$

and

$$\left\{ \begin{array}{lclclcl} 40x_1 & + & 20x_2 & + & 60x_3 & \leq & 1200 & \text{(pounds of R1)} \\ 4x_1 & + & x_2 & + & 6x_3 & \leq & 300 & \text{(pounds of R2)} \\ .2x_1 & + & .7x_2 & + & 2x_3 & \leq & 40 & \text{(hours of labor)} \\ 100x_1 & + & 100x_2 & + & 800x_3 & \leq & 8000 & \text{(area in m}^2\text{)} \\ .1x_1 & + & .3x_2 & + & .8x_3 & \leq & 8 & \text{(machine).} \end{array} \right. \quad (ii)$$

5. (Transportation problem [Vas03, Ex. 2.4]). Another concern that manufacturers face daily is *transportation costs* for their products. Let us look at the following hypothetical situation and try to set it up as a linear program. A manufacturer of widgets has warehouses in Atlanta, Baltimore, and Chicago. The warehouse in Atlanta has 50 widgets in stock, the warehouse in Baltimore has 30 widgets in stock, and the warehouse in Chicago has 50 widgets in stock. There are retail stores in Detroit, Eugene, Fairview, Grove City, and Houston. The retail stores in Detroit, Eugene, Fairview, Grove City, and Houston need at least 25, 10, 20, 30, 15 widgets, respectively. Obviously, the manufacturer needs to ship widgets from the three warehouses and he wants to do this in the cheapest possible way. This presents a perfect backdrop for a linear program to minimize shipping cost. To start, we need to know the cost of shipping one widget from each warehouse to each retail store. This is given by a shipping cost table.

	1.D	2.E	3.F	4.G	5.H
1. Atlanta	55	30	40	50	40
2. Baltimore	35	30	100	45	60
3. Chicago	40	60	95	35	30

Thus, it costs \$30 to ship one unit of the product from Baltimore to Eugene (E), \$95 from Chicago to Fairview (F), and so on.

In order to set this up as a linear program, we introduce variables that represent the number of units of product shipped from each warehouse to each store. We have numbered the warehouses according to their alphabetical order and we have enumerated the stores similarly. Let x_{ij} , for all $1 \leq i \leq 3$, $1 \leq j \leq 5$, represent the number of widgets shipped from warehouse # i

to store # j . This gives us 15 unknowns. The objective function (the quantity to be minimized) is the shipping cost given by

$$\begin{aligned} C = & 55x_{11} + 30x_{12} + 40x_{13} + 50x_{14} + 40x_{15} \\ & + 35x_{21} + 30x_{22} + 100x_{23} + 45x_{24} + 60x_{25} \\ & + 40x_{31} + 60x_{32} + 95x_{33} + 35x_{34} + 30x_{35}, \end{aligned}$$

where $55x_{11}$ represents the cost of shipping one widget from the warehouse in Atlanta to the retail store in Detroit (D) multiplied by the number of widgets that will be shipped, and so forth.

What are the constraints? First, our 15 variables satisfy the condition that

$$x_{ij} \geq 0, \quad \text{for all } 1 \leq i \leq 3, 1 \leq j \leq 5, \quad (i)$$

since shipping a negative amount of widgets makes no sense. Second, since the warehouse # i cannot ship more widgets than it has in stock, we get

$$\begin{cases} x_{11} + x_{12} + x_{13} + x_{14} + x_{15} \leq 50 \\ x_{21} + x_{22} + x_{23} + x_{24} + x_{25} \leq 30 \\ x_{31} + x_{32} + x_{33} + x_{34} + x_{35} \leq 50. \end{cases} \quad (ii)$$

Next, working with the amount of widgets that each retail store needs, we obtain the following five constraints:

$$\begin{cases} x_{11} + x_{21} + x_{31} \geq 25 \\ x_{12} + x_{22} + x_{32} \geq 10 \\ x_{13} + x_{23} + x_{33} \geq 20 \\ x_{14} + x_{24} + x_{34} \geq 30 \\ x_{15} + x_{25} + x_{35} \geq 15. \end{cases} \quad (iii)$$

The problem is now set up. It is a linear program with 15 variables and 23 linear constraints.

6. (Job assignment problem [Vas03, Ex. 2.5]). Suppose that a production manager must assign n workers to do n jobs. If every worker could perform each job at the same level of skill and efficiency, the job assignments could be issued arbitrarily. However, as we know, this is seldom the case. Thus, each of the n workers is evaluated according to the time he or she takes to perform each job. The time, given in hours, is expressed as a number greater than or equal to zero. Obviously, the goal is to assign workers to jobs in such a way that the total time is as small as possible. In order to set up the notation, we let c_{ij} be the time it takes for worker # i to perform job # j . Then the times could naturally be written in a table. For example, take $n = 3$ and let the times be given as in the following table:

	a	b	c
A	10	70	40
B	20	60	10
C	10	20	90

We can examine all six assignments and find that the minimum value of the total time is 40. So, we conclude that the production manager would be wise to assign worker **A** to job **a**, worker **B** to job **c**, and worker **C** to job **b**.

In general, this method of selection is not good. The total number of possible ways of assigning jobs is $n! = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1$. This is an enormous number even for moderate n . For $n = 70$,

$$\begin{aligned} n! = & 119785716699698917960727837216890987364589381425 \\ & 464258575536286462800958278984531968000000000000000. \end{aligned}$$

It has been estimated that if a Sun Workstation computer had started solving this problem at the time of the Big Bang, by looking at all possible job assignments, then by now it would not yet have finished its task.

Although is not obvious, the job assignment problem (with any number n of workers and jobs) can be expressed as a linear program. Namely, we set

$$x_{ij} = 0 \quad \text{or} \quad 1 \quad (i)$$

depending on whether the worker i is assigned to do the job j . The total time is then

$$\sum_i \sum_j c_{ij} x_{ij} \text{ (to be minimized).} \quad (ii)$$

The condition that every worker i is assigned to exactly one job is

$$\sum_j x_{ij} = 1 \quad \text{for all } i. \quad (iii)$$

The condition that exactly one worker is assigned to every job j is

$$\sum_i x_{ij} = 1 \quad \text{for all } j. \quad (iv)$$

The constraints (i) are not linear. If we replace them by linear constraints $x_{i,j} \geq 0$, then we obtain a linear program with n^2 variables and $n^2 + 2n$ linear constraints. Mathematically, it is a transportation problem with every demand and supply equal 1. As such, it can be solved by the simplex method (see below). (When $n = 70$ it takes seconds.)

The simplex method for transportation problem does not involve any divisions. Therefore, an optimal solution it gives integral values for all components $x_{i,j}$. Thus, the conditions (i) hold, and the simplex method solves the job assignment problem. (Another way to express this is that all vertices of the feasible region (iii) to (iv) satisfy (i).)

50.3 Standard and Canonical Forms for Linear Programs

Definitions:

LP in **canonical form** [Vas03] is $\mathbf{c}^T \mathbf{x} + d \rightarrow \min, \mathbf{x} \geq 0, A\mathbf{x} \leq \mathbf{b}$, where $\mathbf{x}$ is a column of distinct (decision) variables, $\mathbf{c}^T$ is a given row, d is a given number, A is a given matrix, and $\mathbf{b}$ is a given column.

LP in **standard form** [Vas03] is $\mathbf{c}^T \mathbf{x} + d \rightarrow \min, \mathbf{x} \geq 0, A\mathbf{x} = \mathbf{b}$, where $\mathbf{x}, \mathbf{c}, d, A$, and $\mathbf{b}$ are as in the previous paragraph.

The **slack variable** for a constraint $f \leq c$ is $s = c - f \geq 0$. The **surplus variable** for a constraint $f \geq c$ is $s = f - c \geq 0$.

Facts:

For background reading on the material in this section, see [Vas03].

1. Every LP can be written in normal as well as standard form using the following five little tricks (*normal* is a term used by some texts):
 - (a) Maximization and minimization problems can be converted to each other. Namely, the problems $f(\mathbf{x}) \rightarrow \min, \mathbf{x} \in S$ and $-f(\mathbf{x}) \rightarrow \max, \mathbf{x} \in S$ are equivalent in the sense that they have the same optimal solutions and $\max = -\min$.
 - (b) The equation $f = g$ is equivalent to the system of two inequalities, $f \geq g, g \leq f$.
 - (c) The inequality $f \leq g$ is equivalent to the inequality $-f \geq -g$.

- (d) The inequality $f \leq g$ is equivalent to $f + x = g, x \geq 0$, where $x = g - f$ is a new variable called a *slack variable*.
 - (e) A variable x unconstrained in sign can be replaced in our program by two new nonnegative variables, $x = x' - x''$, where $x', x'' \geq 0$.
2. The same tricks are sufficient for rewriting any linear program in different standard, canonical, and normal forms used in different textbooks and software packages. In most cases, all decision variables in these forms are assumed to be nonnegative.

Examples:

1. The canonical form $\mathbf{c}^T \mathbf{x} + d \rightarrow \min, \mathbf{x} \geq 0, A\mathbf{x} \leq \mathbf{b}$ can be rewritten in the following standard form:

$$\mathbf{c}^T \mathbf{x} + d \rightarrow \min, \mathbf{x} \geq 0, \mathbf{y} \geq 0, A\mathbf{x} + \mathbf{y} = \mathbf{b}.$$

2. The standard form $\mathbf{c}^T \mathbf{x} + d \rightarrow \min, \mathbf{x} \geq 0, A\mathbf{x} = \mathbf{b}$ can be rewritten in the following canonical form:

$$\mathbf{c}^T \mathbf{x} + d \rightarrow \min, \mathbf{x} \geq 0, A\mathbf{x} \leq \mathbf{b}, -A\mathbf{x} \leq -\mathbf{b}.$$

3. The diet problem (Example 2 in Section 50.2) can be put in canonical form by replacing (ii) with

$$\begin{cases} -0.3a & - & 1.2b & - & 0.7c & - & 3.5d & - & 5.5e & \geq & -50 \\ -73a & - & 96b & - & 2025c & - & 890d & - & 279e & \geq & -4000 \\ -9.6a & + & 7b & - & 19c & - & 57d & - & 22e & \geq & -1000. \end{cases} \quad (ii)$$

4. The blending problem (Example 3 in Section 50.2) is in standard form.

50.4 Standard Row Tableaux

Definitions:

A **standard row tableau** (of [Vas03]) is

$$\begin{array}{cc} \mathbf{x}^T & 1 \\ \left[\begin{array}{cc} A & \mathbf{b} \\ \mathbf{c}^T & d \end{array} \right] & = \mathbf{u} \\ & \rightarrow \min, \mathbf{x} \geq 0, \mathbf{u} \geq 0 \end{array} \quad (\text{SRT})$$

with given matrix A , columns $\mathbf{b}$ and $\mathbf{c}$, and number d , where all decision variables in $\mathbf{x}, \mathbf{u}$ are distinct.

This tableau means the following linear program:

$$A\mathbf{x} + \mathbf{b} = \mathbf{u} \geq 0, \mathbf{x} \geq 0, \mathbf{c}^T \mathbf{x} + d \rightarrow \min.$$

The **basic solution** for the standard tableau (SRT) is $\mathbf{x} = 0, \mathbf{u} = \mathbf{b}$. The corresponding value for the objective function is d .

A standard tableau (SRT) is **row feasible** if $\mathbf{b} \geq 0$, i.e., the basic solution is feasible. Graphically, a feasible tableau looks like

$$\begin{array}{cc} \oplus & 1 \\ \left[\begin{array}{cc} * & \oplus \\ * & * \end{array} \right] & = \oplus \\ & \rightarrow \min, \end{array}$$

where $\oplus$ stands for nonnegative entries or variables.

A standard tableau (SRT) is **optimal** if $\mathbf{b} \geq \mathbf{0}$ and $\mathbf{c} \geq \mathbf{0}$. Graphically, an optimal tableau looks like

$$\begin{array}{cc} \oplus & 1 \\ \left[\begin{array}{cc} * & \oplus \\ \oplus & * \end{array} \right] & = \oplus \\ & \rightarrow \min. \end{array}$$

A **bad row** in a standard tableau is a row of the form $[\ominus -] = \oplus$, where $\ominus$ stands for nonpositive entries, $-$ stands for a negative number, and $\oplus$ stands for a nonnegative variable.

A **bad column** in a standard tableau is a column of the form $\begin{bmatrix} \oplus \\ \oplus \\ - \end{bmatrix}$, where $\oplus$ stands for a nonnegative variable and nonnegative numbers and $-$ stands for a negative number.

Facts:

For background reading on the material in this section see [Vas03].

1. Standard row tableaux are used in simplex method; see Section 50.6 below.
2. The canonical form above can be written in a standard tableau as follows:

$$\begin{array}{cc} \mathbf{x}^T & 1 \\ \left[\begin{array}{cc} -A & \mathbf{b} \\ \mathbf{c}^T & d \end{array} \right] & = \mathbf{u} \\ & \rightarrow \min, \mathbf{x} \geq 0, \mathbf{u} \geq 0 \end{array}$$

where $\mathbf{u} = \mathbf{b} - A\mathbf{x} \geq 0$.

3. The standard form above can be transformed into canonical form $\mathbf{c}^T \mathbf{x} + d \rightarrow \min, \mathbf{x} \geq 0, -A\mathbf{x} \leq -\mathbf{b}, A\mathbf{x} \leq \mathbf{b}$, which gives the following standard tableau:

$$\begin{array}{cc} \mathbf{x}^T & 1 \\ \left[\begin{array}{cc} -A & \mathbf{b} \\ A & -\mathbf{b} \\ \mathbf{c}^T & d \end{array} \right] & = \mathbf{u} \\ & \rightarrow \min, \mathbf{x} \geq 0; \mathbf{u}, \mathbf{v} \geq 0. \end{array}$$

To get a smaller standard tableau, we can solve the system of linear equations $A\mathbf{x} = \mathbf{b}$. If there are no solutions, the linear program is infeasible. Otherwise, we can write the answer in the form $\mathbf{y} = B\mathbf{z} + \mathbf{b}'$, where the column $\mathbf{y}$ contains some variables in $\mathbf{x}$ and the column $\mathbf{z}$ consists of the rest of variables. This gives the standard tableau

$$\begin{array}{cc} \mathbf{z}^T & 1 \\ \left[\begin{array}{cc} B & \mathbf{b}' \\ \mathbf{c}'^T & d' \end{array} \right] & = \mathbf{y} \\ & \rightarrow \min, \mathbf{y} \geq 0, \mathbf{z} \geq 0, \end{array}$$

where $\mathbf{c}'^T \mathbf{z} + d'$ is the objective function $\mathbf{c}^T \mathbf{x} + \mathbf{b}$ expressed in the terms of $\mathbf{z}$.

4. The basic solution of an optimal tableau is optimal.
5. An optimal tableau allows us to describe all optimal solutions as follows. The variables on top with nonzero last entries in the corresponding columns must be zeros. Crossing out these columns and the last row, we obtain a system of linear constraints on the remaining variables describing the set of all optimal solutions. In particular, the basic solution is the only optimal solution if all entries in the $\mathbf{c}$ -part are positive.
6. A bad row shows that the linear program is infeasible.
7. A bad column in a feasible tableau shows that the linear program is unbounded.

Examples:

1. The linear programs in Example 1 of Section 50.2 and written in an SRT and in an SCT in Example 1 of Section 50.8 below.
2. Consider the blending problem, Example 3 in Section 50.2. We solve the system of linear equations for a, b and the objective function f and, hence, obtain the standard tableau

$$\begin{array}{ccc} & c & d & 1 \\ \left[\begin{array}{ccc} 1 & 2 & -0.5 \\ -2 & -3 & 1.5 \\ 0.1 & 0.1 & 1.5 \end{array} \right] & = & \begin{array}{l} a \\ b \\ f \rightarrow \min. \end{array} \end{array}$$

The tableau is not optimal and has no bad rows or columns. The associated LP can be solved graphically, working in the (c, d) -plane. In Example 1 of Section 50.6, we will solve this LP by the simplex method.

50.5 Pivoting

Definitions:

Given a system of linear equations $\mathbf{y} = \mathbf{A}\mathbf{z} + \mathbf{b}$ solved for m variables, a **pivot step** solves it for a subset of m variables which differs from $\mathbf{y}$ in one variable.

Variables in $\mathbf{y}$ are called **basic variables**. The variables in $\mathbf{z}$ are called **nonbasic variables**.

Facts:

For background reading on the material in this section see [Vas03].

1. Thus, a pivot step switches a basic and a nonbasic variable. In other words, one variable leaves the basis and another variable enters the basis.
2. Here is the pivot rule:

$$\begin{array}{cc} x & y \\ \left[\begin{array}{cc} \alpha^* & \beta \\ \gamma & \delta \end{array} \right] = \begin{array}{l} u \\ v \end{array} & \mapsto \begin{array}{cc} u & y \\ \left[\begin{array}{cc} 1/\alpha & -\beta/\alpha \\ \gamma/\alpha & \delta - \beta\gamma/\alpha \end{array} \right] = \begin{array}{l} x \\ v \end{array} \end{array}$$

We switch the two variables x and u . The pivot entry α marked by $*$ must be nonzero, β represents any entry in the pivot row that is not the pivot entry, γ represents any entry in the pivot column that is not the pivot entry, and δ represents any entry outside the pivot row and column.

3. A way to solve any of linear equations $\mathbf{A}\mathbf{x} = \mathbf{b}$ is to write it in the row tableau

$$\begin{array}{c} \mathbf{x}^T \\ [\mathbf{A}] = \mathbf{b} \end{array}$$

and move as many constants from the right margin to the top margin by pivot steps. After this we drop the rows that read $c = c$ with a constant c . If one of the rows reads $c_1 = c_2$ with distinct constants c_1, c_2 , the system is infeasible. The terminal tableau has no constants remaining at the right margin. If no rows are left, the answer is $0 = 0$ (every $\mathbf{x}$ is a solution). If no variables are left at the right margin, we obtain the unique solution $\mathbf{x} = \mathbf{b}'$. Otherwise, we obtain the answer in the form $\mathbf{y} = \mathbf{B}\mathbf{z} + \mathbf{b}'$ with nonempty disjoint sets of variables $\mathbf{y}, \mathbf{z}$.

This method requires somewhat more computations than the Gauss elimination, but it solves the system with parametric $\mathbf{b}$.

Examples:

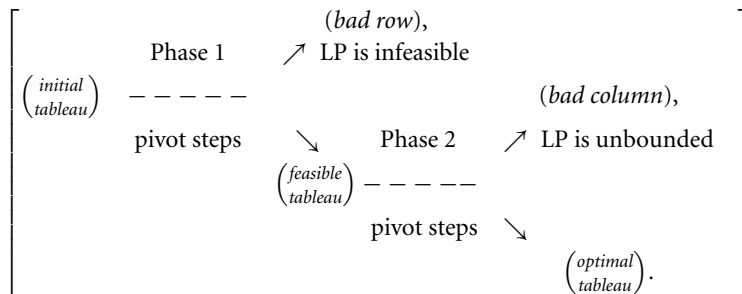
1. Here is a way to solve the system by two pivot steps:

$$\begin{cases} x + 2y = 3 \\ 4x + 7y = 5 \end{cases}$$

$$\begin{array}{cc} x & y \\ \left[\begin{array}{cc} 1^* & 2 \\ 4 & 7 \end{array} \right] = 3 \\ & = 5 \end{array} \mapsto \begin{array}{cc} 3 & y \\ \left[\begin{array}{cc} 1 & -2 \\ 4 & -1^* \end{array} \right] = x \\ & = 5 \end{array} \mapsto \begin{array}{cc} 3 & 5 \\ \left[\begin{array}{cc} -7 & 2 \\ 4 & -1 \end{array} \right] = x \\ & = y', \end{array}$$

50.6 Simplex Method

Now we give the simplex method in terms of standard tableaux. The method consists of finitely many pivot steps, separated in two phases (stages). In Phase 1, we obtain either a feasible tableau or a bad row. In Phase 2, we work with feasible tableaux. We obtain either a bad column or an optimal tableau. Following is the scheme of the simplex method, where LP stands for the linear program:



Phase 2 of simplex method. We start with a feasible tableau (SRT).

1. Is the tableau optimal? If yes, we write the answer: $\min = d$ at $\mathbf{x} = 0, \mathbf{u} = \mathbf{b}$.
2. Are there any bad columns? If yes, the linear program is unbounded.
3. (Choosing a pivot entry) We choose a negative entry in the $\mathbf{c}$ -part, say, c_j in column j . Then we consider all negative entries $a_{i,j}$ above to choose our pivot entry. For every $a_{i,j} < 0$ we compute $b_i/a_{i,j}$ where b_i is the last entry in the row i . Then we maximize $b_i/a_{i,j}$ to obtain our pivot entry $a_{i,j}$.
4. Pivot and go to Substep 1.

Phase 1 of simplex method. We start with a standard tableau (SRT).

1. Is the tableau feasible? If yes, go to Phase 2.
2. Are there bad rows? If yes, the LP is infeasible.
3. (Choosing a pivot entry) Let the first negative number in the **b**-part be in the row i . Consider the subtableau consisting of the first i rows with the i th row multiplied by -1 , and choose the pivot entry as in Substep 3 of Phase 2.
4. Pivot and go to Substep 1.

A pivot step (in Phase 1 or 2) is **degenerate** if the last entry in the pivot row is 0, i.e., the (basic) feasible solution stays the same.

A **cycle** in the simplex method (Phase 1 or 2) is a finite sequence of pivot steps which starts and ends with the same tableau.

Facts:

For background reading on the material in this section see [Vas03].

1. In Phase 2, the current value of the objective function (the last entry in the last row) either improves (decreases) or stays the same (if and only if the pivot step is degenerate).
2. In Phase 1, the first negative entry in the last column increases or stays the same (if and only if the pivot step is degenerate).
3. Following this method, we either terminate in finitely many pivot steps or, after a while, all our steps are degenerate, i.e., the basic solution does not change and we have a cycle.
4. The basic solutions in a cycle are all the same.
5. To prevent cycling, we can make a perturbation (small change in the **b**-part) such that none of the entries in this part of the tableau is ever 0 (see [Vas03] for details).
6. Another approach was suggested by Bland. We can make an ordered list of our variables, and then whenever there is a choice we choose the variable highest on the list (see [Vas03] for a reference and the proof that this rule prevents cycling). Bland's rule also turns the simplex method into a deterministic algorithm, i.e., it eliminates freedom of choices allowed by simplex method. Given an initial tableau and an ordered list of variables, Bland's rule dictates a unique finite sequence of pivot steps resulting in a terminal tableau.
7. With Bland's rule, the simplex method (both phases) terminates in at most $\binom{m+n}{n} - 1$ pivot steps where m is the number of basis variables and n is the number of nonbasic variables (so the tableau has $m+1$ rows and $n+1$ columns). The number $\binom{m+n}{n}$ here is an upper bound for the number of all standard tableaux that can be obtained from the initial tableau by pivot steps, up to permutation of the variables on the top (between themselves) and permutation of the variables at the right margin (between themselves).
8. If all data for an LP are rational numbers, then all entries of all standard tableaux are rational numbers. In particular, all basic solutions are rational. If the LP is feasible and bounded, then there is an optimal solution in rational numbers. However, like for a system of linear equations, the numerators and denominators could be so large that finding them could be impractical.

Examples:

1. Consider the blending problem, Example 3 in Section 50.2.

We start with the standard tableau in Example 2 of Section 50.3. The simplex method allows two choices for the first pivot entry, namely, 1 or 2 in the first row. We pick 1 as the pivot entry:

$$\begin{array}{ccc|l} c & d & 1 & \\ \hline 1^* & 2 & -0.5 & = a \\ -2 & -3 & 1.5 & = b \\ 0.1 & 0.1 & 1.5 & = f \rightarrow \min \end{array} \quad \mapsto \quad \begin{array}{ccc|l} a & d & 1 & \\ \hline 1 & -2 & 0.5 & = c \\ -2 & 1 & 0.5 & = b \\ 0.1 & -0.1 & 1.55 & = f \rightarrow \min . \end{array}$$

Now the tableau is feasible, and we can proceed with Phase 2:

$$\begin{array}{ccc} a & d & 1 \\ \left[\begin{array}{ccc} 1 & -2^* & 0.5 \\ -2 & 1 & 0.5 \\ 0.1 & -0.1 & 1.55 \end{array} \right] & = \begin{array}{l} c \\ b \\ f \rightarrow \min \end{array} & \mapsto \begin{array}{ccc} a & c & 1 \\ \left[\begin{array}{ccc} 0.5 & -0.5 & 0.25 \\ -1.5 & -0.5 & 0.75 \\ 0.15 & 0.05 & 1.525 \end{array} \right] & = \begin{array}{l} d \\ b \\ f \rightarrow \min \end{array} \end{array} .$$

The tableau is optimal, so $\min = 1.525$ at $a = 0, b = 0.75, c = 0$, and $d = 0.25$.

50.7 Geometric Interpretation of Phase 2

Definitions:

A subset S of $\mathbf{R}^N$ is **closed** if S contains the limit of any convergent sequence in S .

A **convex linear combination** or **mixture** of two points $\mathbf{x}, \mathbf{y}$, is $\alpha\mathbf{x} + (1 - \alpha)\mathbf{y}$, where $0 \leq \alpha \leq 1$. A particular case of a mixture is the **halfsum** $\mathbf{x}/2 + \mathbf{y}/2$.

The **line segment** connecting distinct points $\mathbf{x}, \mathbf{y}$ consists of all mixtures of $\mathbf{x}, \mathbf{y}$.

A set S is **convex** if it contains all mixtures of its points.

An **extreme point** in a convex set is a point that is not the halfsum of any two distinct points in the set, i.e., the set stays convex after removing the point.

In the case of a closed convex set with finitely many extreme points, the extreme points are called **vertices**.

Two extreme points in a convex set are **adjacent** if the set stays convex after deleting the line segment connecting the points.

Facts:

For background reading on the material in this section see [Vas03].

1. The feasible region for any linear program is a closed convex set with finitely many extreme points (vertices).
2. Consider a linear program and the associated feasible tableaux. The vertices of the feasible region are the basic solutions of the feasible tableaux. Permutation of rows or columns does not change the basic solution.
3. A degenerate pivot step does not change the basic solution. Any nondegenerate pivot step takes us from a vertex to an adjacent vertex with a better value for the objective function.
4. The set of optimal solutions for any linear program is a closed convex set with finitely many extreme points.
5. The number of pivot steps in Phase 2 without cycles (say, with Bland's rule) is less than the number of vertices in the feasible region.
6. For (SRT) with $m + 1$ rows and $n + 1$ columns, the number of vertices in the feasible region is at most $\binom{m+n}{m}$. When $n = 1$, this upper bound can be improved to 2. When $n = 2$, this upper bound can be improved to $m + 2$. It is unknown (in 2005) whether, for arbitrary m, n , a bound exists that is a polynomial in $m + n$.
7. The total number of pivot steps in both phases (with Bland's rule) is less than $\binom{m+n}{m}$.

50.8 Duality

Definitions:

A **standard column tableau** has the form

$$\begin{array}{c} -y \\ 1 \end{array} \left[\begin{array}{cc} A & b \\ \mathbf{c}^T & d \end{array} \right] \quad y \geq 0, \mathbf{v} \geq 0 \quad (\text{SCT})$$

$\parallel$ $\downarrow$
 $\mathbf{v}^T$ $\max.$

The **associated LP** is $-\mathbf{y}^T \mathbf{b} + d \rightarrow \max, -\mathbf{y}^T A + \mathbf{c}^T = \mathbf{v}^T \geq 0, \mathbf{y} \geq 0$.

The **basic solution** associated with (SCT) is $\mathbf{y} = 0, \mathbf{v} = \mathbf{c}$.

The tableau (SCT) is **column feasible** if $\mathbf{c} \geq 0$, i.e., the basic solution is feasible. (So, a tableau is optimal if and only if it is both row and column feasible.)

The linear program in (SCT) is **dual** to that in (SRT). We can write both as the row and the column problem with the same matrix:

$$\begin{array}{rcl} & \mathbf{x}^T & 1 \\ -\mathbf{y} \left[\begin{array}{cc} A & \mathbf{b} \\ \mathbf{c}^T & d \end{array} \right] = \mathbf{u} & \mathbf{x} \geq 0, \mathbf{u} \geq 0 \\ 1 \left[\begin{array}{cc} \mathbf{c}^T & d \end{array} \right] = z \rightarrow \min & \mathbf{y} \geq 0, \mathbf{v} \geq 0 \\ = \mathbf{v}^T = w \rightarrow \max. & & \end{array} \quad (\text{ST})$$

Facts:

For background reading on the material in this section, see [Vas03].

1. The linear program in the standard row tableau (SRT) can be written in the following standard column tableau:

$$\begin{array}{rcl} -\mathbf{x} & \left[\begin{array}{cc} -A^T & \mathbf{c} \\ \mathbf{b}^T & -d \end{array} \right] & \mathbf{x} \geq 0, \mathbf{u} \geq 0 \\ 1 & & \\ & \parallel & \downarrow \\ & \mathbf{u}^T & \max. \end{array}$$

Then the dual problem becomes the row problem with the same matrix. This shows that the dual of the dual is the primal program.

2. The pivot rule for column and row tableaux is the same inside tableaux:

$$\begin{array}{rcl} -x \left[\begin{array}{cc} \alpha^* & \beta \\ \gamma & \delta \end{array} \right] & \rightarrow & -u \left[\begin{array}{cc} 1/\alpha & -\beta/\alpha \\ \gamma/\alpha & \delta - \beta\gamma/\alpha \end{array} \right] \\ \parallel & & \parallel \\ u & v & x \quad v \end{array}$$

3. If a linear program has an optimal solution, then the simplex method produces an optimal tableau. This tableau is also optimal for the dual program, hence both programs have the same optimal values (the duality theorem).
4. The duality theorem is a deep fact with several interpretations and applications. Geometrically, the duality theorem means that certain convex sets can be separated from points outside them by hyperplanes. We will see another interpretation of the duality theorem in the theory of matrix games (see below). For problems in economics, the dual problems and the duality theorem also have important economic interpretations (see examples below).
5. If a linear program is unbounded, then (after writing it in a standard row tableau) the simplex method produces a row feasible tableau with a bad column. The bad column shows that the dual problem is infeasible.
6. There is a standard tableau that has both a bad row and a bad column, hence there is an infeasible linear program such that the dual program is also infeasible.
7. Here is another way to express the duality theorem: Given a feasible solution for a linear program and a feasible solution for the dual problem, they are both optimal if and only if the feasible values are the same.
8. Given a feasible solution for a linear program and a feasible solution for the dual problem, they are both optimal if and only if for every pair of dual variables at least one value is 0, i.e., the *complementary slackness* condition $\mathbf{v}^T \mathbf{x} + \mathbf{y}^T \mathbf{u} = 0$ in terms of (ST) holds.
9. More precisely, given a feasible solution $(\mathbf{x}, \mathbf{u})$ for the row program in (ST) and a feasible solution $(\mathbf{y}, \mathbf{v})$ for the column program, the difference $(\mathbf{c}^T \mathbf{x} + d) - (-\mathbf{y}^T \mathbf{u} + d)$ between the corresponding feasible values is $\mathbf{v}^T \mathbf{x} + \mathbf{y}^T \mathbf{u}$.

10. All pairs $(\mathbf{x}, \mathbf{u})$, $(\mathbf{y}, \mathbf{v})$ of optimal solutions for the row and column programs in (ST) are described by the following system of linear constraints: $A\mathbf{x} + \mathbf{b} = \mathbf{u} \geq 0$, $\mathbf{x} \geq 0$, $-\mathbf{y}^T A + \mathbf{c} = \mathbf{v} \geq 0$, $\mathbf{y} \geq 0$, $\mathbf{v}^T \mathbf{x} + \mathbf{y}^T \mathbf{u} = 0$. This is a way to show that solving a linear program can be reduced to finding a feasible solution for a finite system of linear constraints.

Examples:

(Generalizations of Examples 1 to 4 in section 50.2 above and their duals):

1. The linear programs $c_1 x_1 + \dots + c_n x_n \rightarrow \max$, all $x_i \geq 0$, $x_1 + \dots + x_n = 1$, and $y \rightarrow \min$, $y \geq c_i$ for all i in Example 1 are dual to each other. To see this, we use the standard tricks: $y = y' - y''$ with $y', y'' \geq 0$; $x_1 + \dots + x_n = 1$ is equivalent to $x_1 + \dots + x_n \leq 1$, $-x_1 - \dots - x_n \leq -1$.

We obtain the following standard tableau:

$$\begin{array}{rcccl} & y' & y'' & 1 & \\ -\mathbf{x} & \left[\begin{array}{ccc} \mathbf{J} & -\mathbf{J} & -\mathbf{c} \end{array} \right] & = \oplus & \\ 1 & \left[\begin{array}{ccc} 1 & -1 & 0 \end{array} \right] & = y \rightarrow \min & \\ & = \oplus & = \oplus & \rightarrow \max, & \end{array}$$

where $\mathbf{J}$ is the column of n ones, $\mathbf{c} = [c_1, \dots, c_n]^T$, and $\mathbf{x} = [x_1, \dots, x_n]^T$.

2. Consider the general diet problem (a generalization of Example 2):

$$A\mathbf{x} \geq \mathbf{b}, \mathbf{x} \geq 0, C = \mathbf{c}^T \mathbf{x} \rightarrow \min,$$

where m variables in $\mathbf{x}$ represent different foods and n constraints in the system $A\mathbf{x} \geq \mathbf{b}$ represent ingredients. We want to satisfy given requirements $\mathbf{b}$ in ingredients using given foods at minimal cost C .

On the other hand, we consider a warehouse that sells the ingredients at prices $y_1, \dots, y_n \geq 0$. Its objective is to maximize the profit $P = \mathbf{y}^T \mathbf{b}$, matching the price for each food: $\mathbf{y}^T A \leq \mathbf{c}^T$.

We can write both problems in a standard tableau using slack variables $\mathbf{u} = A\mathbf{x} - \mathbf{b} \geq 0$ and $\mathbf{v}^T = \mathbf{c}^T - \mathbf{y}^T A \geq 0$:

$$\begin{array}{rcccl} & \mathbf{x}^T & 1 & & \\ -\mathbf{y} & \left[\begin{array}{cc} A & -\mathbf{b} \end{array} \right] & = \mathbf{u} & \mathbf{x} \geq 0, \mathbf{u} \geq 0 & \\ 1 & \left[\begin{array}{cc} \mathbf{c}^T & 0 \end{array} \right] & = C \rightarrow \min & \mathbf{y} \geq 0, \mathbf{v} \geq 0 & \\ & = \mathbf{v}^T & = P & \rightarrow \max. & \end{array}$$

So, these two problems are dual to each other. In particular, the simplex method solves both problems and if both problems are feasible, then $\min(C) = \max(P)$.

The optimal prices for the ingredients in the dual problem are also called *dual* or *shadow* prices. These prices tell us how the optimal value reacts to small changes in $\mathbf{b}$; see Section 50.9 below.

3. Consider the general mixing problem (a generalization of Example 3):

$$A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq 0, C = \mathbf{c}^T \mathbf{x} \rightarrow \min,$$

where m variables in $\mathbf{x}$ represent different alloys and n constraints in $A\mathbf{x} = \mathbf{b}$ represent elements. We want to satisfy given requirements $\mathbf{b}$ in elements using given alloys at minimal cost C .

On the other hand, consider a dealer who buys and sells the elements at prices $y_1, \dots, y_n$. A positive price means that the dealer sells, and negative price means that the dealer buys. Dealer's objective is to maximize the profit $P = \mathbf{y}^T \mathbf{b}$ matching the price for each alloy: $\mathbf{y}^T A \leq \mathbf{c}$.

To write the problems in standard tableaux, we use the standard tricks and artificial variables:

$$\begin{array}{l} \mathbf{u}' = A\mathbf{x} - \mathbf{b} \geq 0, \mathbf{u}'' = -A\mathbf{x} + \mathbf{b} \geq 0; \\ \mathbf{v}^T = \mathbf{c}^T - \mathbf{y}^T A \geq 0; \mathbf{y} = \mathbf{y}' - \mathbf{y}'', \mathbf{y}' \geq 0, \mathbf{y}'' \geq 0. \end{array}$$

Now we manage to write both problems in the same standard tableau:

$$\begin{array}{rcl} & \mathbf{x}^T & 1 \\ -\mathbf{y}' & \left[\begin{array}{cc} A & -\mathbf{b} \end{array} \right] & = \mathbf{u}' \\ -\mathbf{y}'' & \left[\begin{array}{cc} -A & \mathbf{b} \end{array} \right] & = \mathbf{u}'' \\ 1 & \left[\begin{array}{cc} \mathbf{c}^T & 0 \end{array} \right] & = C \rightarrow \min \\ & = \mathbf{v}^T = P & \rightarrow \max. \end{array} \quad \begin{array}{l} \mathbf{x} \geq 0; \mathbf{u}', \mathbf{u}'' \geq 0 \\ \mathbf{y}', \mathbf{y}'' \geq 0; \mathbf{v} \geq 0 \end{array}$$

4. Consider a generalization of the manufacturing problem in Example 4:

$$P = \mathbf{c}^T \mathbf{x} \rightarrow \max, \quad A\mathbf{x} \leq \mathbf{b}, \quad \mathbf{x} \geq 0,$$

where the variables in $\mathbf{x}$ are the amounts of products, P is the profit (or revenue) you want to maximize, constraints $A\mathbf{x} \leq \mathbf{b}$ correspond to resources (e.g., labor of different types, clean water you use, pollutants you emit, scarce raw materials), and the given column $\mathbf{b}$ consists of amounts of resources you have. Then the dual problem

$$\mathbf{y}^T \mathbf{b} \rightarrow \min, \quad \mathbf{y}^T A \geq \mathbf{c}^T, \quad \mathbf{y} \geq 0$$

admits the following interpretation. Your competitor, Bob, offers to buy you out at the following terms: You go out of business and he buys all resources you have at price $\mathbf{y}^T \geq 0$, matching your profit for every product you may want to produce, and he wants to minimize his cost.

Again Bob's optimal prices are your resource shadow prices by the duality theorem. The shadow price for a resource shows the increase in your profit per unit increase in the quantity b_0 of the resource available or decrease in the profit when the limit b_0 decreases by one unit. While changing b_0 , we do not change the limits for the other resources and any other data for our program. There are only finitely many values of b_0 for which the downward and upward shadow prices are different. One of these values could be the borderline between the values of b_0 for which the corresponding constraint is binding or nonbinding (in the sense that dropping of this constraint does not change the optimal value).

The shadow price of a resource cannot increase when supply b_0 of this resource increases (the law of diminishing returns, see the next section).

5. (General transportation problem and its dual). We have m warehouses and n retail stores. The warehouse # i has a_i widgets available and the store # j needs b_j widgets.

It is assumed that the following *balance condition* holds:

$$\sum_{i=1}^m a_i = \sum_{j=1}^n b_j.$$

If total supply is greater than total demand, the problem can be reduced to the one with the balance condition by introducing a fictitious store where the surplus can be moved at zero cost. If total supply is less than total demand, the program is infeasible.

The cost of shipping a widget from warehouse # i to store # j is denoted by c_{ij} and the number of widgets shipped from warehouse # i to store # j is denoted by x_{ij} . The linear program can be stated as follows:

$$\left\{ \begin{array}{l} \text{minimize} \quad C(x_{11}, \dots, x_{mn}) = \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} \\ \text{subject to} \quad \sum_{i=1}^m x_{ij} \geq b_j, \quad j = 1, \dots, n \\ \sum_{j=1}^n x_{ij} \leq a_i, \quad i = 1, \dots, m \\ x_{ij} \geq 0, \quad i = 1, \dots, m, \quad j = 1, \dots, n. \end{array} \right.$$

The dual program is

$$-\sum_{i=1}^m a_i u_i + \sum_{j=1}^n b_j v_j \rightarrow \max, w_{ij} = c_{ij} + u_i - v_j \geq 0 \quad \text{for all } i, j; u_i \geq 0, v_j \geq 0.$$

So, what is a possible meaning of the dual problem? The control variables u_i, v_j of the dual problem are called *potentials* or “zones.” While the potentials correspond to the constraints on each retail store and each warehouse (or to the corresponding slack variables), there are other variables w_{ij} in the dual problem that correspond to the decision variable x_{ij} of the primal problem.

Imagine that you want to be a mover and suggest a simplified system of tariffs. Instead of mn numbers $c_{i,j}$, we use $m + n$ “zones.” Namely, you assign a “zone” $u_i \geq 0$ $i = 1, 2$ to each of the warehouses and a “zone” $v_j \geq 0$ $j = 1, 2$ to each of the retail stores. The price you charge is $v_j - u_i$ instead of $c_{i,j}$. To beat competition, you want $v_j - u_i \leq c_{i,j}$ for all i, j . Your profit is $-\sum a_i u_i + \sum b_j v_j$ and you want to maximize it.

The simplex method for any transportation problem can be implemented using m by n tables rather than $mn + 1$ by $m + n + 1$ tableaux. Since all pivot entries are ± 1 , no division is used. In particular, if all a_i, b_j are integers, we obtain an optimal solution with integral $x_{i,j}$.

Phase 1 is especially simple. If $m, n \geq 2$, we choose any position and write down the maximal possible number (namely, the minimum of supply and demand). Then we cross out the row or column and adjust the demand or supply respectively. If $m = 1 < n$, we cross out the column. If $n = 1 < m$, we cross out the row. If $m = n = 1$, then we cross out both the row and column. Thus, we find a feasible solution in $m + n - 1$ steps, which correspond to pivot steps, and the $m + n - 1$ selected positions correspond to the basic variables.

Every transportation problem with the balance condition has an optimal solution. (See [Vas03] for details.)

50.9 Sensitivity Analysis and Parametric Programming

Sensitivity analysis is concerned with how small changes in data affect the optimal value and optimal solutions, while large changes are studied in parametric programming.

Consider the linear program given by (SRT) with the last column being an affine function of a parameter t , i.e., we replace b, d by affine functions $b + b_1 t, d + d_1 t$ of a parameter t . Then the optimal value becomes a function $f(t)$ of t .

Facts:

For background reading on the material in this section, see [Vas03].

1. The function $f(t)$ is defined on a closed convex set S on the line, i.e., S is one of the following “intervals”: empty set, a point, an interval $a \leq t \leq b$ with $a < b$, a ray $t \geq a$, a ray $t \leq a$, the whole line.
2. The function $f(t)$ is piece-wise linear, i.e., the interval S is a finite union of subintervals S_k with $f(t)$ being an affine function on each subinterval.
3. The function $f(t)$ is *convex* in the sense that the set of points in the plane below the plot is a convex set. In particular, the function $f(t)$ is continuous.
4. In parametric programming, there are methods for computing $f(t)$. The set S is covered by a finite set of tableaux optimal for various values of t . The number of these tableaux is at most $\binom{m+n}{m}$.
5. Suppose that the LP with $t = 0$ (i.e., the LP given by (SRT)) has an optimal tableau T_0 . Let $\mathbf{x} = \mathbf{x}^{(0)}, \mathbf{u} = \mathbf{u}^{(0)}$ be the corresponding optimal solution (the basic solution), and let $\mathbf{y} = \mathbf{y}^{(0)}, \mathbf{v} = \mathbf{v}^{(0)}$ be the corresponding optimal solution for the dual problem (see (ST) for notation). Assume that the $\mathbf{b}$ -part of T_0 has no zero entries. Then the function $f(t)$ is affine in an interval containing 0. Its slope, i.e., derivative $f'(0)$ at $t = 0$, is $f'(0) = d_1 + \mathbf{b}_1^T \mathbf{v}^{(0)}$. Thus, we can easily compute $f'(0)$

from T_0 . In other words, the optimal tableaus give the partial derivatives of the optimal value with respect to the components of given $\mathbf{b}$ and d .

6. If we pivot the tableau with a parameter, the last column stays an affine function of the parameter and the rest of the tableau stays independent of parameter.
7. Similar facts are true if we introduce a parameter into the last row rather than into the last column.
8. If both the last row and the last column are affine functions of parameter t , then after pivot steps, the A -part stays independent of t , the $\mathbf{b}$ -part and $\mathbf{c}$ -part stay affine functions of t , but the d -part becomes a quadratic polynomial in t . So the optimal value is a piece-wise quadratic function of t .
9. If we want to maximize, say, profit (rather than minimize, say, cost), then the optimal value is concave (convex upward), i.e., the set of points below the graph is convex. The fact that the slope is nonincreasing is referred to as the *law of diminishing returns*.

50.10 Matrix Games

Matrix games are very closely related with linear programming.

Definitions:

A matrix game is given by a matrix A , the **payoff matrix**, of real numbers.

There are two players. The players could be humans, teams, computers, or animals. We call them *He* and *She*. He chooses a row, and She chooses a column. The corresponding entry in the matrix represents what she pays to him (the payoff of the row player). Games like chess, football, and blackjack can be thought of as (very large) matrix games. Every row represents a strategy, i.e., his decision what to do in every possible situation. Similarly, every column corresponds to a strategy for her. The rows are his (**pure**) **strategies** and columns are her (pure) strategies.

A **mixed strategy** is a mixture of pure strategies. In other words, a mixed strategy for him is a probability distribution on the rows and a mixed strategy for her is a probability distribution on the columns.

We write his mixed strategy as columns $\mathbf{p} = (p_i)$ with $\mathbf{p} \geq 0$, $\sum p_i = 1$. We write her mixed strategy as rows $\mathbf{q}^T = (q_j)$ with $\mathbf{q} \geq 0$, $\sum q_j = 1$.

The corresponding payoff is $\mathbf{p}^T A \mathbf{q}$, the mathematical expectation.

A pair of strategies (his strategy, her strategy) is an **equilibrium** or a **saddle point** if neither player can gain by changing his or her strategy. In other words, no player can do better by a unilateral change. (The last sentence can be used to define the term “equilibrium” in any game, while “saddle point” is used only for zero-sum two-player games; sometimes it is restricted to the pure joint strategies.)

In other words, at an equilibrium, the payoff $\mathbf{p}^T A \mathbf{q}$ has maximum as function of $\mathbf{p}$ and minimum as function of $\mathbf{q}$.

His mixed strategy $\mathbf{p}$ is **optimal** if his worst case payoff $\min(\mathbf{p}^T A)$ is maximal. The maximum is the **value of the game** (for him). Her mixed strategy $\mathbf{q}^T$ is **optimal** if her worst case payoff $\min(-A \mathbf{q})$ is maximal. The maximum is the value of the game for her.

If the payoff matrix is skew-symmetric, the matrix game is called **symmetric**.

Facts:

For background reading on the material in this section, see [Vas03].

1. A pair (i, j) of pure strategies is a saddle point if and only if the entry $a_{i,j}$ of the payoff matrix $A = a_{i,j}$ is both largest in its column and the smallest in its row.
2. Every matrix game has an equilibrium.
3. A pair (his strategy, her strategy) is an equilibrium if and only if both strategies are optimal.
4. His payoff $\mathbf{p}^T A \mathbf{q}$ at any equilibrium $(\mathbf{p}, \mathbf{q})$ equals his value of the game and equals the negative of her value of the game (the *minimax theorem*).
5. To solve a matrix game means to find an equilibrium and the value of the game.

6. Solving a matrix game can be reduced to solving the following dual pair of linear programs written in a standard tableau:

$$\begin{array}{rcccl}
 & \mathbf{q}^T & \mu' & \mu'' & 1 \\
 -\mathbf{p} & \left[\begin{array}{cccc} -A & \mathbf{1}_m & -\mathbf{1}_m & 0 \end{array} \right] & = & * & \geq 0 \\
 -\lambda' & \left[\begin{array}{cccc} \mathbf{1}_n^T & 0 & 0 & -1 \end{array} \right] & = & * & \geq 0 \\
 -\lambda'' & \left[\begin{array}{cccc} -\mathbf{1}_n^T & 0 & 0 & 1 \end{array} \right] & = & * & \geq 0 \\
 1 & \left[\begin{array}{cccc} 0 & 1 & -1 & 0 \end{array} \right] & = & \mu & \rightarrow \min \\
 & \parallel & \parallel & \parallel & \parallel \\
 & * & * & * & \lambda \rightarrow \max.
 \end{array}$$

Here, A is the m by n payoff matrix, $\mathbf{1}_n^T$ is the row of n ones, $\mathbf{1}_m$ is the column of m ones, $\mathbf{p}$ is his mixed strategy, $\mathbf{q}^T$ is her mixed strategy. Note that her problem is the row problem and his problem is the column problem. Their problems are dual to each other. Since both problems have feasible solutions (take, for example, $\mathbf{p} = \mathbf{1}_m/m$, $\mathbf{q} = \mathbf{1}_n/n$), the duality theorem says that $\min(f) = \max(g)$. That is,

his value of the game $= \max(\lambda) = \min(\mu) = -$ her value of the game.

Thus, the minimax theorem follows from the duality theorem.

7. We can save two rows and two columns and get a row feasible tableau as follows:

$$\begin{array}{rcccl}
 & \mathbf{q}/(\mu + c) & 1 \\
 -\mathbf{p}/(\lambda + c) & \left[\begin{array}{cc} -A - c\mathbf{1}_m\mathbf{1}_n^T & \mathbf{1}_m \\ -\mathbf{1}_n^T & 0 \end{array} \right] & = & * & \geq 0 \\
 1 & & = & -1/(\mu + c) & \rightarrow \min \\
 & \parallel & \parallel \\
 & * & -1/(\lambda + c) & \rightarrow \max.
 \end{array}$$

Here we made sure that the value of game is positive by adding a number c to all entries of A , i.e., replacing A by $A + c\mathbf{1}_m\mathbf{1}_n^T$. E.g., $c = 1 - \min(A)$. Again his and her problems are dual to each other, since they share the same standard tableau. Since the tableau is feasible, we bypass Phase 1.

8. An arbitrary dual pair (ST) of linear programs can be reduced to a symmetric matrix game, with the following payoff matrix

$$M = \begin{bmatrix} 0 & -A & -\mathbf{b} \\ A^T & 0 & -\mathbf{c} \\ \mathbf{b}^T & \mathbf{c}^T & 0 \end{bmatrix}.$$

Its size is $(m + n + 1) \times (m + n + 1)$, where $m \times n$ is the size of the matrix A .

9. The definition of equilibria makes sense for any game (not only for two-player zero-sum games). However, finding equilibria is not the same as solving the game when there are equilibria with different payoffs or when cooperation between players is possible and makes sense.
10. For any symmetric game, the value is 0, and the optimal strategies for the row and column players are the transposes of each other.

Examples:

1. [Vas03, Ex. 19.3] Solve the matrix game

$$A = \begin{bmatrix} 5 & 0 & 6 & 1 & -2 \\ 2 & 1 & 2 & 1 & 2 \\ -9 & 0 & 5 & 2 & -9 \\ -9 & -8 & 0 & 4 & 2 \end{bmatrix}.$$

We mark the maximal entries in every column by *. Then we mark the minimal entries in each row by '. The positions marked by both * and ' are exactly the saddle points:

$$A = \begin{bmatrix} 5^* & 0 & 6^* & 1 & -2' \\ 2 & 1^{*'} & 2 & 1' & 2^* \\ -9' & 0 & 5 & 2 & -9' \\ -9' & -8 & 0 & 4^* & 2^* \end{bmatrix}.$$

In this example, the position $(i, j) = (2, 2)$ is the only saddle point. The corresponding payoff (the value of the game) is 1.

2. This small matrix game is known as Heads and Tails or Matching Pennies. We will call the players *He* and *She*. He chooses: heads (*H*) or tails (*T*). Independently, she chooses: *H* or *T*. If they choose the same, he pays her a penny. Otherwise, she pays him a penny. Here is his payoff in cents:

$$\begin{array}{cc} & \text{She} \\ & \begin{array}{cc} H & T \end{array} \\ \begin{array}{c} He \\ H \\ T \end{array} & \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \end{array}.$$

There is no equilibrium in pure strategies. The only equilibrium in mixed strategies is $([1/2, 1/2]^T, [1/2, 1/2])$. The value of game is 0. The game is not symmetric in the usual sense. However the game is symmetric in the following sense: if we switch the players and also switch *H* and *T* for a player, then we get the same game.

3. Another game is Rock, Scissors, Paper. In this game two players simultaneously choose *Rock*, *Scissors*, or *Paper*, usually by a show of hand signals on the count of three, and the payoff function is defined by the rules *Rock breaks Scissors*, *Scissors cuts Paper*, *Paper covers Rock*, and every strategy ties against itself. Valuing a win at 1, a tie at 0, and a loss at -1 , we can represent the game with the following matrix, where, for both players, strategy 1 is *Rock*, strategy 2 is *Scissors*, and strategy 3 is *Paper*:

$$A = \begin{bmatrix} 0 & 1 & -1 \\ -1 & 0 & 1 \\ 1 & -1 & 0 \end{bmatrix}.$$

The only optimal strategy for the column player is $\mathbf{q}^T = [1/3, 1/3, 1/3]$. Since the game is symmetric, the value is 0, and $\mathbf{q}$ is the only optimal strategy for the row player.

50.11 Linear Approximation

Definitions:

An l^p -**best linear approximation** (fit) of a given column $\mathbf{w}$ with n entries by the columns of a given m by n matrix A is $A\mathbf{x}$, where $\mathbf{x}$ is an optimal solution for $\|\mathbf{w} - A\mathbf{x}\|_p \rightarrow \min$. (See Chapter 37 for information about $\|\cdot\|_p$.)

In other words, we want the vector $\mathbf{w} - A\mathbf{x}$ of **residuals** (offsets, errors) to be smallest in a certain sense.

Facts:

For background reading on the material in this section, see [Vas03].

1. Most common values for p are 2, 1, ∞ . In statistics, usually $p = 2$ and the first column of the matrix A is the column of ones. In simple regression analysis, $n = 2$. In multiple regression, $n \geq 3$.

In time series analysis, the second column of A is an arithmetic progression representing time; typically, this column is $[1, 2, \dots, m]^T$. (See Chapter 52 for more information.)

2. If $n = 1$ and the matrix A is a column of ones, we want to approximate given numbers w_i by one number Y . When $p = 2$, the best fit is the arithmetic mean $(\sum w_i)/m$. When $p = 1$, the best fits are the medians. When $p = \infty$, the best fit is the midrange $(\min(w_i) + \max(w_i))/2$.
3. When $p = 2$, the l^2 -norm is the usual Euclidean norm, the most common way to measure the size of a vector, and this norm is used in Euclidean geometry. The best fit is known as the least squares fit. To find it, we drop a perpendicular from w onto the column space of A . In other words, we want the vector $w - Ax$ to be orthogonal to all columns of A ; that is, $A^T(w - Ax) = 0$. This gives a system of n linear equations $A^T Ax = A^T w$ for n unknowns in the column x . The system always has a solution. Moreover, the best fit Ax is the same for all solutions x . In the case when w belongs to the column space, the best fit is w (this is true for all p). Otherwise, x is unique. (See Section 5.8 or Chapter 39 for more information about least squares methods.)
4. The best l^∞ -fit is also known as the least-absolute-deviation fit and the Chebyshev approximation.
5. When $p = 1$, finding the best fit can be reduced to a linear program. Namely, we reduce the optimization problem with the objective function $\|e\|_1 \rightarrow \min$, where $e = (e_i) = w - Ax$, to a linear program using m additional variables u_i such that $|e_i| \leq u_i$ for all i . We obtain the following linear program with $m + n$ variables a_j, u_i and $2m$ linear constraints:

$$\sum u_i \rightarrow \min, \quad -u_i \leq w_i - A_i x \leq u_i \quad \text{for } i = 1, \dots, m,$$

where A_i is the i th row of the given matrix A .

6. When $p = \infty$, finding the best fit can also be reduced to a linear program. Namely reduce the optimization problem with the objective function $\|e\|_\infty \rightarrow \min$, where $e = (e_i) = w - Ax$, to a linear program using an additional variable u such that $|e_i| \leq u$ for all i . A similar trick was used when we reduced solving matrix games to linear programming. We obtain the following linear program with $n + 1$ variables a_j, u and $2m$ linear constraints:

$$t \rightarrow \min, \quad -u \leq w_i - A_i x \leq u \quad \text{for } i = 1, \dots, m,$$

where A_i is the i th row of the given matrix A .

7. Any linear program can be reduced to finding a best l^∞ -fit.

Examples:

1. [Vas03, Prob. 22.7] Find the best l^p -fit $w = ch^2$ for $p = 1, 2, \infty$ given the following data:

i	1	2	3
Height h in m	1.6	1.5	1.7
Weight w in kg	65	60	70

Compare the optimal values for c with those for the best fits of the form $w/h^2 = c$ with the same p (the number w/h^2 in kg/m^2 is known as BMI, the body mass index). Compare the minimums with those for the best fits of the form $w = b$ with the same p .

Solution

Case $p = 1$. We could convert this problem to a linear program with four variables and then solve it by the simplex method (see Fact 6 above). But we can just solve graphically the nonlinear program with the objective function

$$f(c) = |65 - 1.6^2 c| + |60 - 1.5^2 c| + |70 - 1.7^2 c|$$

to be minimized and no constraints. The function $f(c)$ is piece-wise affine and convex. The optimal solution is $c = x_1 = 65/1.6^2 \approx 25.39$. This c equals the median of the three observed BMIs $65/1.6^2, 60/1.5^2$ and $70/1.7^2$. The optimal value is

$$\min = |65 - c_1 1.6^2| + |60 - c_1 1.5^2| + |70 - c_1 1.7^2| = 6.25.$$

To compare this with the best l^1 -fit for the model $w = b$, we compute the median $b = x_1 = 65$ and the corresponding optimal values:

$$\min = |65 - x_1| + |60 - x_1| + |70 - x_1| = 10.$$

So the model $w = ch^2$ is better than $w = b$ for our data with the l^1 -approach.

Case $p = 2$. Our optimization problem can be reduced to solving a linear equation for c (see Fact 3 above). Also we can solve the problem using calculus, taking advantage of the fact that our objective function

$$f(c) = (65 - 1.6^2 c)^2 + (60 - 1.5^2 c)^2 + (70 - 1.7^2 c)^2$$

is differentiable. The optimal solution is $c = x_2 = 2518500/99841 \approx 25.225$. This x_2 is not the mean of the observed BMIs, which is about 25.426. The optimal value is $\min \approx 18$.

The mean of w_i is 65, and the corresponding minimal value is $5^2 + 0^2 + 5^2 = 50$. So, again the model $w = ch^2$ is better than $w = b$.

Case $p = \infty$. We could reduce this problem to a linear program with two variables and then solve it by graphical method or simplex method (see Fact 7 above). But we can do a graphical method with one variable. The objective function to minimize now is

$$f(c) = \max(|65 - 1.6^2 c|, |60 - 1.5^2 c|, |70 - 1.7^2 c|).$$

This objective function $f(c)$ is piecewise affine and convex. The optimal solution is

$$c_\infty = 6500/257 \approx 25.29.$$

It differs from the midrange of the BMIs, which is about 25.44. The optimal value is ≈ 3 .

On the other hand, the midrange of the weights w_i is 65, which gives $\min = 5$ for the model $w = b$ with the best l^∞ -fit. So, again the model $w = ch^2$ is better than $w = b$.

2. [Vas03, Ex. 2, p. 255] A student is interested in the number w of integer points $[x, y]$ in the disc $x^2 + y^2 \leq r^2$ of radius r . He computed w for some r :

r		1	2	3	4	5	6	7	8	9
w		5	13	29	45	81	113	149	197	253

The student wants to approximate w by a simple formula $w = ar + b$ with constants a, b . But you feel that the area of the disc, πr^2 would be a better approximation and, hence, the best l^2 -fit of the form $w = ar^2$ should work even better for the numbers above.

Compute the best l^2 -fit (the least squares fit) for both models, $w = ar + b$ and $w = ar^2$, and find which is better. Also compare both optimal values with

$$\sum_{i=1}^9 (w_i - \pi i^2)^2.$$

Solution

For our data, the equation $w = ar + b$ is the system of linear equations $A\mathbf{x} = \mathbf{w}$, where

$$\mathbf{w} = [5, 13, 29, 45, 81, 113, 149, 197, 253]^T,$$

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}^T, \text{ and } \mathbf{x} = \begin{bmatrix} a \\ b \end{bmatrix}.$$

The least-squares solution is the solution to $A^T A \mathbf{x} = A^T \mathbf{w}$, i.e.,

$$\begin{bmatrix} 285 & 45 \\ 45 & 9 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} 6277 \\ 885 \end{bmatrix}.$$

So, $a = 463/15, b = -56$. The error

$$\sum_{i=1}^9 (w_i - 463i/15 + 56)^2 = 48284/15 \approx 3218.93.$$

For $w = ar^2$, the system is $Ba = w$, where w is as above and $B = [1^2, 3^2, 3^2, 4^2, 5^2, 6^2, 7^2, 8^2, 9^2]^T$. The equation $B^T Ba = B^T w$ is $415398a = 47598$, hence $a = 23799/7699 \approx 3.09118$. The error

$$\sum_{i=1}^9 (w_i - 23799i^2/7699)^2 = 2117089/7699 \approx 274.982.$$

So, the model $w = ar^2$ gives a much better least-squares fit than the model $w = ar + b$ although it has one parameter instead of two.

The model $w = \pi r^2$ without parameters gives the error

$$\sum_{i=1}^9 (w_i - \pi i)^2 = 147409 - 95196\pi + 15398\pi^2 \approx 314.114.$$

It is known that $w_i/h_i^2 \rightarrow \pi$ as $i \rightarrow \infty$. Moreover, $|w_i - \pi r_i^2|/r_i$ is bounded as $i \rightarrow \infty$. It follows that $a \rightarrow \pi$ for the the best L_p -fit $w = ar^2$ for any $p \geq 1$ as we use data for $r = 1, 2, \dots, n$ with $n \rightarrow \infty$.

50.12 Interior Point Methods

Definitions:

A point $\mathbf{x}$ in a convex subset $X \subset \mathbf{R}^n$ is **interior** if

$$\{\mathbf{x} + (\mathbf{x} - \mathbf{y})\delta/\|\mathbf{y} - \mathbf{x}\|_2 : \mathbf{y} \in X, \mathbf{y} \neq \mathbf{x}\} \subset X$$

for some $\delta > 0$.

The **boundary** of X is the points in X , which are not interior.

An **interior solution** for a linear program is an interior point of the feasible region.

An **interior point method** for solving bounded linear programs starts with given interior solutions and produces a sequence of interior solutions which converges to an optimal solution.

An **exterior point method** for linear programs starts with a point outside the feasible region and produces a sequence of points which converges to feasible solution (in the case when the LP is feasible).

Facts:

For background reading on the material in this section see [Vas03].

1. In linear programming, the problem of finding a feasible solution (i.e., Phase 1) and the problem of finding an optimal solution starting from a feasible solution (i.e., Phase 2) can be reduced to each other. So, the difference between interior point methods and exterior point methods is not so sharp.
2. The simplex method, Phase 1, can be considered as an exterior point method which (theoretically) converges in finitely many steps. The ellipsoid method [Ha79] is truly an exterior point method. Hačijan proved an exponential convergence for the method.

3. In the simplex method, Phase 2, we travel from a vertex to an adjacent vertex and reach an optimal vertex (if the LP is bounded) in finitely many steps. The vertices are not interior solutions unless the feasible region consists of single point. If an optimal solution for a linear program is interior, then all feasible solutions are optimal.
4. The first interior point method was given by Brown in the context of matrix games. The convergence was proved by J. Robinson. J. von Neumann suggested a similar method with better convergence.
5. Karmarkar suggested an interior point method with exponential convergence. After him, many similar methods were suggested. They can be interpreted as follows. We modify our objective function by adding a penalty for approaching the boundary. Then we use Newton's method. The recent progress is related to understanding of properties of convex functions which make minimization by Newton's method efficient. Recent interior methods beat simplex methods for very large problems.
6. [Vas03, Sec. A8] On the other hand, it is known that for LPs with small numbers of variables (or, by duality, with a small number of constraints), there are faster methods than the simplex method.

For example, consider an (SRT) with only two variables on top and m variables at the right margin (basis variables). The feasible region S has at most $m + 2$ vertices. Phase 2, starting with any vertex, terminates in at most $m + 1$ pivot steps.

At each pivot step, it takes at most two comparisons to check whether the tableau is optimal or to find a pivot column. Then in m sign checks, at most m divisions, and at most $m - 1$ comparisons we find a pivot entry or a bad column.

Next, we pivot to compute the new $3m + 3$ entries of the tableau. In one division, we find the new entry in the pivot row that is not the last entry (the last entry was computed before) and in $2m$ multiplications and $2m$ additions, we find the new entries outside the pivot row and column. Finally, we find the new entries in the pivot column in $m + 1$ divisions. So a pivot step, including finding a pivot entry and pivoting, can be done in $8m + 3$ operations — arithmetic operations and comparisons.

Thus, Phase 2 can be done in $(m + 1)(8m + 3)$ operations. While small savings in this number are possible (e.g., at the $(m + 1)$ -th pivot step we need to compute only the last column of the tableau), it is unlikely that any substantial reduction of this number for any modification of the simplex method can be achieved (in the worst case). Concerning the number of pivot steps, for any $m \geq 1$ it is clearly possible for S to be a bounded convex $(m + 2)$ -gon, in which case for any vertex there is a linear objective function such that the simplex method requires exactly $\lfloor 1 + n/2 \rfloor$ pivot steps with only one choice of pivot entry at each step. It is also possible to construct an $(m + 2)$ -gon, an objective point, and an initial vertex such that m pivot steps, with unique choice of pivot entry at each step, are required. There is an example with two choices at the first step such that the first choice leads to the optimal solution while the second choice leads to m additional pivot steps with unique choice.

Using a fast median search instead of the simplex method, it is possible to have Phase 2 done in $\leq 100m + 100$ operations.

References

- [Ha79] L.G. Hačijan, A polynomial algorithm in linear programming. (Russian) *Dokl. Akad. Nauk SSSR* 244 (1979), 5, 1093–1096.
- [Ros00] K.H. Rosen, Ed., *Handbook of Discrete and Combinatorial Mathematics*, CRC Press, Boca Raton, FL, 2000.
- [Vas03] L.N. Vaserstein (in collaboration with C.C. Byrne), *Introduction to Linear Programming*, Prentice-Hall, Upper Saddle River, NJ, 2003.

51

Semidefinite Programming

Henry Wolkowicz
University of Waterloo

51.1	Introduction	51-1
51.2	Specific Notation and Preliminary Results	51-3
51.3	Geometry	51-5
51.4	Duality and Optimality Conditions	51-5
51.5	Strong Duality without a Constraint Qualification...	51-7
51.6	A Primal-Dual Interior-Point Algorithm	51-8
51.7	Applications of SDP	51-9
	References	51-11

51.1 Introduction

Semidefinite programming, SDP, refers to optimization problems where variables X in the objective function and/or constraints can be symmetric matrices restricted to the cone of positive semidefinite matrices. (We restrict ourselves to real symmetric matrices, $\mathcal{S}_n$, since the vast majority of applications are for the real case. The complex case requires using the complex inner-product space.) An example of a *simple linear SDP* is

$$\begin{aligned} p^* = & \min && \text{tr } CX \\ \text{(SDP)} & \text{subject to} && TX = \mathbf{b} \\ & && X \succeq 0, \end{aligned}$$

where $T : \mathcal{S}_n \rightarrow \mathbb{R}^m$. The details are given in the definitions in section 51.2; the SDP relaxation of the Max-Cut problem is given in Example 1 in this section. The linear SDP is a generalization of the standard *linear programming problem* (see Chapter 50), $(\text{LP}) \min \mathbf{c}^T \mathbf{x}$, s.t. $A\mathbf{x} = \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$, where $A \in \mathbb{R}^{m \times n}$, and the element-wise nonnegativity constraint $\mathbf{x} \geq \mathbf{0}$ is replaced by the positive semidefinite constraint, $X \succeq 0$. These cone constraints are the *hard constraints* for these linear problems, i.e., they introduce a combinatorial element into the problem. In the LP case, this refers to finding the *active constraints*, i.e., finding the *active set* at the optimum $\{j : x_j^* = 0\}$. For SDP, this translates to finding the set of zero eigenvalues of the optimum X^* . However, for SDP this is further complicated by the unknown eigenvectors.

But there are surprisingly many parallels between semidefinite and linear programming as well as interesting differences. The parallels include: elegant and strong duality theory, many important applications, and the successful interior-point approaches for handling the hard constraints. The differences include possible existence of duality gaps, as well as possible failure of strict complementarity. Since LP is such a well-known area, we emphasize the similarities and differences between LP and SDP as we progress through this chapter.

The study of SDP, or linear matrix inequalities (LMI), dates back to the work of Lyapunov on the stability of trajectories from differential equations in 1890, [Lya47]. More recently, applications in control theory appear in the work of [Yak62]. More details are given in [BEF94].

Cone Programming also called *generalized linear programming*, is a generalization of SDP [BF63]. Other books dealing with problems over cones that date back to the 1960s are [Lue69], [Hol75], and [Jah86]. More recently, SDP falls into the class of *symmetric or homogeneous cone programming* [Fay97], [SA01], which includes optimization over combinations of: the nonnegative orthant; the cone of positive semidefinite matrices, and the Lorentz (or second-order) cone.

Positive definite and Euclidean matrix completion problems are feasibility questions in SDP. Research dates back to the early 1980s [DG81], [GJM84]. This continues to be an active area of research [Lau98], [Joh90]. (More recently, positive definite completion theory is being used to solve large scale instances of SDP [BMZ02], [FKM01].)

Combinatorial optimization applications fueled the popularity of SDP in the 1980s, [Lov79] and the strong approximation results in [GW95] and Example 1 in this section. A related survey paper is [Ren99]. SDP continues to attract new applications in, e.g., molecular conformation [AKW99], sensor localization [Jin05], [SY06], and optimization over polynomials [HL03].

The fact that SDP is a convex program that can be solved to any desired accuracy in polynomial time follows from the seminal work in [NN94].

Examples:

1. Suppose that we are given the weighted undirected graph $G = (V, E)$ with vertex set $V = \{1, \dots, n\}$ and nonnegative edge weights w_{ij} , $ij \in E$ (with $w_{ij} = 0$, $ij \notin E$). The Max-Cut problem finds the partition of the vertices into two parts so as to maximize the sum of the weights on the edges that are cut (vertices in different parts). This problem arises in, e.g., physics and VLSI design. We can model this problem as a ± 1 program with quadratic functions.

$$\begin{aligned} \text{(MC)} \quad \mu^* = & \max \quad \frac{1}{4} \sum_{ij=1}^n w_{ij} (1 - x_i x_j) = \frac{1}{4} \mathbf{x}^T L \mathbf{x} \\ & \text{subject to} \quad x_j^2 = 1, \quad j = 1, \dots, n, \end{aligned}$$

where L is the Laplacian matrix of the graph, $L_{ij} = \begin{cases} \sum_{k \neq i} w_{ik} & \text{if } i = j; \\ -w_{ij} & \text{if } i \neq j. \end{cases}$ We consider the simple example with vertex set $V = \{1, 2, 3\}$ and nonnegative edge weights $w_{12} = 1, w_{13} = 2, w_{23} = 2$.

Here, the Laplacian matrix $L = \begin{bmatrix} 3 & -1 & -2 \\ -1 & 3 & -2 \\ -2 & -2 & 4 \end{bmatrix}$. The optimal solution places vertices 1 and 2

in one part and vertex 3 in the other; thus it cuts the edges 13 and 23. An optimal solution is

$$\mathbf{x}^* = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix} \text{ with optimal value } \mu^* = 4.$$

The well known semidefinite relaxation for the Max-Cut problem is obtained using the commutativity of the trace $\mathbf{x}^T L \mathbf{x} = \text{tr } L \mathbf{x} \mathbf{x}^T = \text{tr } L X$ and discarding the (hard) rank one constraint on X .

$$\begin{aligned} \text{(MCSDP)} \quad \mu^* \leq \nu^* = & \max \quad \frac{1}{4} \text{tr } L X \\ & \text{subject to} \quad \text{diag}(X) = \mathbf{e} \\ & X \succeq 0, \end{aligned}$$

where the linear transformation $\text{diag}(X) : \mathcal{S}_n \rightarrow \mathbb{R}^n$ denotes the diagonal of X , and $\mathbf{e}$ is the vector

of ones. The optimal solution to MCSDP is $X^* = \begin{bmatrix} 1 & 1 & -1 \\ 1 & 1 & -1 \\ -1 & -1 & 1 \end{bmatrix}$. (This is verified using duality; see Example 3 in section 51.4 below.) In fact, here X^* is rank one, and column one of X^* provides

the optimal solution $\mathbf{x}^*$ of MC and the optimal value $\mu^* = \nu^* = 4$.

For this simple example, X^* was rank one and the SDP relaxation obtained the exact solution of the NP-hard MC problem. This is not true in general. However, the MCSDP relaxation is a surprisingly strong relaxation for MC. A randomized approximation algorithm based on MCSDP provides a ± 1 vector $\mathbf{x}$ with objective value at least .87856 times the optimal value μ^* [GW94]. Empirical tests obtain even stronger results.

51.2 Specific Notation and Preliminary Results

Note that in this section all matrices are real.

Definitions:

For general $m \times n$ matrices $M, N \in \mathbb{R}^{m \times n}$, we use the matrix inner product $\langle M, N \rangle = \text{tr } N^T M$. The corresponding norm is $\|M\|_F = \sqrt{\text{tr } M^T M}$, the **Frobenius norm**.

$\mathcal{S}_n$ denotes the space of **symmetric** $n \times n$ **matrices**.

The set of **positive semidefinite (respectively definite) matrices** is denoted by PSD (respectively, PD); $X \succeq 0$ means $X \in \text{PSD}$.

The **linear SDP** is

$$\begin{aligned} \text{(SDP)} \quad p^* = \quad & \min \quad \text{tr } CX \\ & \text{subject to} \quad TX = \mathbf{b} \\ & \quad \quad \quad X \succeq 0, \end{aligned}$$

where the variable $X \in \mathcal{S}_n$, the linear transformation $T : \mathcal{S}_n \rightarrow \mathbb{R}^m$, the vector $\mathbf{b} \in \mathbb{R}^m$, and the matrix $C \in \mathcal{S}_n$ are (given) problem parameters. The constraint $TX = \mathbf{b}$ can be written concretely as $\text{tr } A_k X = b_k$, for some given $A_k \in \mathcal{S}_n$, $k = 1, \dots, m$. Thus, SDP can be expressed explicitly as

$$\begin{aligned} \text{(SDP)} \quad p^* = \quad & \min \quad \sum_{ij} C_{ij} X_{ij} \\ & \text{subject to} \quad \sum_{ij} A_{ijk} X_{ij} = b_k, \quad k = 1, \dots, m \\ & \quad \quad \quad X \succeq 0, \end{aligned}$$

where A_{ijk} denotes the ij element of the symmetric matrix A_k .

For $M \in \mathbb{R}^{m \times n}$, $\text{vec}(M) \in \mathbb{R}^{mn}$ is the vector formed from the columns of M .

For $S \in \mathcal{S}_n$, $\text{svec}(S) \in \mathbb{R}^{\frac{n(n+1)}{2}}$ is the vector formed from the upper triangular part of S , taken column-wise, with the strict upper triangular part multiplied by $\sqrt{2}$. The inverse of svec is denoted sMat .

For $U \in \mathcal{S}_n$, the **symmetric Kronecker product** is defined by

$$(M \otimes N) \text{svec}(U) = \text{svec} \left(\frac{1}{2} (\text{NUM}^T + \text{MUN}^T) \right).$$

For a linear transformation between two vector spaces $T : V \rightarrow W$, the **adjoint linear transformation** is denoted $T^{\text{adj}} : W \rightarrow V$, i.e., it is the unique linear transformation that satisfies the adjoint equation for the inner products in the respective vector spaces

$$\langle T\mathbf{v}, \mathbf{w} \rangle_W = \langle \mathbf{v}, T^{\text{adj}} \mathbf{w} \rangle_V, \quad \forall \mathbf{v} \in V, \mathbf{w} \in W.$$

If $V = W$, then this reduces to the definition of the **adjoint of a linear operator** given in Chapter 5.3.

We follow the standard notation used in SDP: for $\mathbf{y} \in \mathbb{R}^n$, $\text{Diag}(\mathbf{y})$ denotes the diagonal matrix formed using the vector $\mathbf{y}$, while the adjoint linear transformation is $\text{diag}(X) = \text{Diag}^{\text{adj}}(X) \in \mathbb{R}^n$.

For an inner product space V , the **polar cone** of $S \subseteq V$ is

$$S^+ = \{\phi : \langle \phi, \mathbf{s} \rangle \geq 0, \forall \mathbf{s} \in S\}.$$

Given a set $K \subseteq \mathbb{R}^n$, we let $\text{cone}(K)$ denote the convex cone generated by K . (See Chapter 9.1 for more information on cones.) A set K is a convex cone if it is closed under nonnegative scalar multiplication and

addition, i.e.,

$$\alpha K \subseteq K, \forall \alpha \geq 0, \quad K + K \subseteq K.$$

For a convex cone K , the convex cone $\mathcal{F} \subseteq K$ is a **face** of K (denoted $\mathcal{F} \triangleleft K$) if

$$\mathbf{x}, \mathbf{y} \in K, \mathbf{z} = \alpha \mathbf{x} + (1 - \alpha) \mathbf{y} \in \mathcal{F}, 0 < \alpha < 1 \quad \text{implies} \quad \mathbf{x}, \mathbf{y} \in \mathcal{F}.$$

Facts:

1. In the space of real, symmetric matrices, $\mathcal{S}_n$, the matrix inner product reduces to $\langle X, Y \rangle = \text{tr } XY$.
2. For general compatible matrices M, N , the trace is commutative, i.e., $\text{tr } MN^T = \text{tr } N^T M = \sum_{i=1}^n \sum_{j=1}^n M_{ij} N_{ij}$.
3. svec (and sMat) is an isometry between $\mathcal{S}_n$ (equipped with the Frobenius norm) and $\mathbb{R}^{\frac{n(n+1)}{2}}$ (equipped with the Euclidean norm). The svec transformation is used in algorithms when solving symmetric matrix equations. Using the isometry (rather than the ordinary vec) provides additional robustness.
4. PSD is a closed convex cone and the interior of PSD consists of the positive definite matrices. The following are equivalent: (a) $A \succeq 0$ ($A \succ 0$), (b) the vector of eigenvalues $\lambda(A) \geq 0$ ($\lambda(A) > 0$), (c) all principal minors ≥ 0 (all leading principal minors > 0).
5. The Kronecker product $K \otimes L$ is easily formed using the blocks $K_{ij} L$. It is useful in changing matrix equations into vector equations (see [HJ94]), (here K is the unknown matrix) $\text{vec}(NKM^T) = (M \otimes N) \text{vec}(K)$. Similarly, the symmetric Kronecker product $M \stackrel{s}{\otimes} N$ changes matrix equations with symmetric matrix variables $U \in \mathcal{S}_n$ to vector equations. (See the definition in section 51.2.) By extending the definition from $\mathcal{S}_n$ to $\mathbb{R}^{n \times n}$, the symmetric Kronecker product can be expressed explicitly using the standard Kronecker product see ([TW05]),

$$8M \stackrel{s}{\otimes} N = (I + A)(N \otimes M^T + M \otimes N^T) + (N \otimes M^T + M \otimes N^T)(I + A),$$

where A is the matrix representation of the transpose transformation. Matrix equations with symmetric matrix variables arise when finding the search direction in interior-point methods for SDP; see section 51.6. Surprisingly (see [TW05]), for $A, B \in \mathcal{S}_n$,

$$A \stackrel{s}{\otimes} B \succeq 0 \iff A \otimes B \succeq 0;$$

$$A \stackrel{s}{\otimes} B \succ 0 \iff A \otimes B \succ 0.$$

6. If $A \in \mathbb{R}^{m \times n}$ and the linear transformation $T\mathbf{v} = A\mathbf{v}$, then $T^{\text{adj}} \cong A^T$.
7. The polar cone is a closed convex cone and the second polar $(K^+)^+ = K$ if and only if K is a closed convex cone.

Examples:

1.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 3 & 5 & 6 \end{bmatrix}, \mathbf{v} = \begin{bmatrix} 1 \\ 2\sqrt{2} \\ 4 \\ 3\sqrt{2} \\ 5\sqrt{2} \\ 6 \end{bmatrix}, \quad \text{svec}(A) = \mathbf{v} \\ \|A\|_F = \|\mathbf{v}\|_2 = \sqrt{129}.$$

2. The polars: (a) $(\mathbb{R}^n)^+ = \{\mathbf{0}\}$; $\{\mathbf{0}\}^+ = \mathbb{R}^n$, (b) for the unit vector $\mathbf{e}_i \in \mathbb{R}^n$, $\{\mathbf{e}_i\}^+ = \{\mathbf{v} \in \mathbb{R}^n : v_i \geq 0\}$, (c) $(\text{cone}\{(\begin{smallmatrix} 1 & 1 \end{smallmatrix})^T, (\begin{smallmatrix} 1 & 0 \end{smallmatrix})^T\})^+ = \text{cone}\{[\begin{smallmatrix} 0 & 1 \end{smallmatrix}]^T, [\begin{smallmatrix} 1 & -1 \end{smallmatrix}]^T\}$; (d) let $\mathcal{I} = \{\mathbf{v} \in \mathbb{R}^3 : v_3^2 \leq v_1 v_2, v_1 \geq 0, v_2 \geq 0\}$ be the so-called ice-cream cone [Ber73], i.e., the cone of vectors that makes an angle at most 45 degrees with the vector $(\begin{smallmatrix} 1 & 1 & 1 \end{smallmatrix})^T$. Then $\mathcal{I} = \mathcal{I}^+$.

51.3 Geometry

Extending LP to SDP can be thought of as replacing a polyhedral cone (the nonnegative orthant $(\mathbb{R}^+)^n$) by the nonpolyhedral cone PSD. We now see that many of the geometric properties follow through from $K = (\mathbb{R}^+)^n$ to $K = \text{PSD}$.

Definitions:

A cone K is called **self-polar** if it equals its polar $K = K^+$.

A face $\mathcal{F} \triangleleft K$ is called **facially exposed** if $\mathcal{F} = K \cap \phi^\perp$, for some $\phi \in K^+$.

A face $\mathcal{F} \triangleleft K$ is called **projectionally exposed** if $\mathcal{F}$ is the image of K under an orthogonal projection, $\mathcal{F} = P(K)$.

Facts:

- Both $(\mathbb{R}^+)^n$ and PSD are closed convex cones with nonempty interior. And both are self-polar cones, i.e., $((\mathbb{R}^+)^n)^+ = (\mathbb{R}^+)^n$, $(\text{PSD})^+ = \text{PSD}$. (See [Ber73].)
- Suppose that $\hat{X} \in \text{relint } \mathcal{F} \subseteq \text{PSD}$ and $\text{rank } \hat{X} = r$. Then $\hat{X} = PD_r P^T$, where P is the $n \times r$ matrix with orthonormal columns of eigenvectors corresponding to nonzero (positive) eigenvalues. We get that $\mathcal{F} \triangleleft \text{PSD}$ if and only if $\mathcal{F} = P\mathcal{S}_+^r P^T$. Equivalently, faces $\mathcal{F} \triangleleft \text{PSD}$ are characterized by $\{Y \in \text{PSD} : \mathcal{R}(Y) \subseteq \mathcal{R}(\hat{X})\}$, where $\hat{X}$ is any matrix in the relative interior of $\mathcal{F}$ and $\mathcal{R}$ denotes the range. Equivalently, we could use $\mathcal{N}(Y) \supset \mathcal{N}(\hat{X})$, where $\mathcal{N}$ denotes nullspace. (See [Boh48], [BC75].)
- Both $(\mathbb{R}^+)^n$ and PSD are facially and projectionally exposed, i.e., all faces are both facially and projectionally exposed [BW81], [BLP87], [ST90].
- The following relates to closure conditions that are needed for strong duality results. Suppose that $\mathcal{F} \triangleleft \text{PSD}$. Then (see [RTW97]),

$\text{PSD} + \mathcal{F}^\perp$ is closed; $\text{PSD} + \text{Span } \mathcal{F}$ is not closed.

Examples:

- Consider the face $\mathcal{F} = \{X \in \text{PSD} : X\mathbf{e} = \mathbf{0}\}$, i.e., the face of PSD of centered matrices, positive semidefinite matrices with row and column sums equal to 0. Let $P = (\mathbf{e} \ V)$ be an orthogonal matrix, i.e., $V^T \mathbf{e} = 0$, $V^T V = I$. Then we get $\mathcal{F} = V\{X \in \mathcal{S}_{n-1} : X \geq 0\}V^T$, i.e., a one-to-one mapping between the face $\mathcal{F}$ and the semidefinite cone in $\mathcal{S}_{n-1}$. This relationship is used in [AKW99] to map Euclidean distance matrices to positive semidefinite matrices of full rank.

51.4 Duality and Optimality Conditions

Duality lies behind efficient algorithms in optimization. Unlike LP, strong duality can fail for SDP if a *constraint qualification* does not hold. We consider duality the linear *primal* SDP introduced above a game-theoretic approach.

Definitions:

The corresponding **Lagrangian function** (or payoff function from primal player X , who plays matrix X , to dual player Y , who plays vector $\mathbf{y}$) is $L(X, \mathbf{y}) = \text{tr } CX + \mathbf{y}^T(\mathbf{b} - TX)$.